

Chapter 9

MPEG Audio

The MPEG-1 audio standard describes an algorithm suitable for coding a monophonic or stereo audio signal with a bandwidth of up to 24 kHz. The algorithm is divided into three levels of coding complexity called *layers*. The Layer I algorithm is the simplest algorithm in terms of computational complexity and requires the highest bit rate to produce an output audio quality that is indistinguishable from the original. The Layers II and III algorithms include additional components that increase the computational complexity of the Layer I algorithm and consequently require lower bit rates to provide indistinguishable quality.

The MPEG-1 audio coding algorithm was finalized as an international standard in 1993 [1], and the algorithms used in the standard are based on two perceptual audio coding algorithms that were developed in the late 1980s. Layer I is a simplified version of the MUSICAM (masking-pattern universal subband integrated coding and multiplexing) algorithm. The MUSICAM algorithm was derived from an earlier subband coding scheme called MASCAM (masking pattern adapted subband coding and multiplexing) and has been adopted in Europe for use in digital audio broadcasting. Layer II is essentially identical to MUSICAM and Layer III is a combination of techniques from MUSICAM and the ASPEC (adaptive spectral perceptual entropy coding) transform-based algorithm.

The MPEG-2 audio standards extend the functionality of the MPEG-1 algorithm to allow lower input sampling rates and the coding of *multichannel* audio signals for surround-sound reproduction. Early psychoacoustic experiments showed that at least four channels are required to produce a realistic surround-sound field. But due to limitations in the technology available, two-channel stereo became the accepted standard for home sound reproduction. Today though, with the improvements in digital coding techniques and the increased capacity of storage devices, multichannel audio systems have become a viable alternative.

In particular, the ITU-R have recommended a five-channel audio system known as 3/2 stereo that consists of left and right channels (L and R), a center channel (C), and two rear surround channels (L_S and R_S) [2]. This system produces a realistic surround-sound field with a stable frontal sound image and a large listening area.

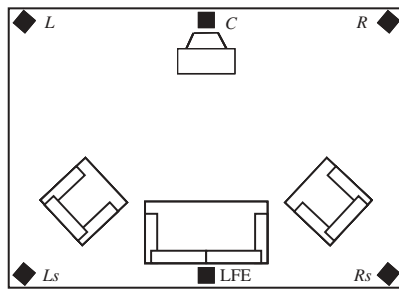


Figure 9.1 A typical speaker position for a 5.1 channel surround-sound listening environment.

An optional low-frequency enhancement channel (LFE) can be added to increase the level of frequency components between 15 and 120 Hz. A 3/2 stereo system combined with a LFE channel is usually referred to as a 5.1 channel system and a typical speaker configuration for such a system in a home listening environment is shown in Figure 9.1.

The MPEG-2 audio coding standard contains two completely separate algorithms. The first algorithm was designed to allow backward compatibility with MPEG-1 and is usually referred to as MPEG-2 BC (backward compatible). It was finalized as an international standard in 1995 with amendment 1 added in 1996 [3]. The second algorithm (known as MPEG-2 AAC (advanced audio coding)) was added as a new part to the MPEG-2 standard in 1997 [4].

The MPEG-2 BC standard uses essentially the same algorithms as the MPEG-1 encoder and decoder, but has added functionality to allow lower input sampling rates for mono and stereo inputs and multichannel audio at the MPEG-1 input sampling rates. The backward compatibility between MPEG-2 BC and MPEG-1 means that an MPEG-2 decoder can decode an MPEG-1 bit stream and an MPEG-1 decoder can derive a stereo signal from an MPEG-2 multichannel bit stream provided that an appropriate matrixing procedure is conducted before encoding. An MPEG-2 encoder may also produce a multichannel bit stream that is not backward compatible with an MPEG-1 decoder.

The MPEG-2 AAC algorithm was designed to include many of the latest audio coding compression techniques that could not be implemented while maintaining backward compatibility. These new compression techniques allow the MPEG-2 AAC coder to produce indistinguishable quality at significantly lower bit rates than MPEG-2 BC.

The DVB digital television standard specifies that an IRD (integrated receiver-decoder) must conform to the following guidelines for audio:

- MPEG-2 Layers I and II is supported by the IRD;
- The use of Layer II is recommended for the encoded bit stream;
- IRDs support single channel, dual channel, joint stereo, stereo, and the extraction of at least a stereo pair from MPEG-2 compatible multichannel audio;

- Sampling rates of 32, 44.1, and 48 kHz are supported by IRDs;
- The encoded bit stream does not use emphasis.

This chapter describes the MPEG-1 and MPEG-2 algorithms and bit-stream syntax associated with these minimum requirements of a DVB IRD.

9.1. MPEG-1 LAYER I,II ENCODERS

The algorithms used in the Layer I encoder and decoder are a subset of the algorithms used in layer II. This section describes the algorithms that are common to the Layers I and II encoders. The extra parts of the standard that are used only in the Layer II encoder are described in Section 9.2.

The MPEG-1 Layer I encoder is intended to be used to code wideband audio signals that have been converted to PCM (pulse code modulation) samples with sampling rates of 32, 44.1, or 48 kHz. The encoder can operate using one of the following four modes:

1. Single channel (a single audio signal)
2. Dual channel (two independent audio signals)
3. Stereo (a left and right stereo pair)
4. Joint stereo (a left and right stereo pair with the stereo redundancy used to reduce the output bit rate).

The Layer I, II encoders produce a digital bit stream with a constant output bit rate that is specified in the bit stream syntax. The maximum output bitrates are 448 and 384 kbits/s for Layers I and II, respectively.

The basic block diagram for a Layer I encoder is shown in Figure 9.2. The remainder of this section contains a very brief description of the algorithm and then

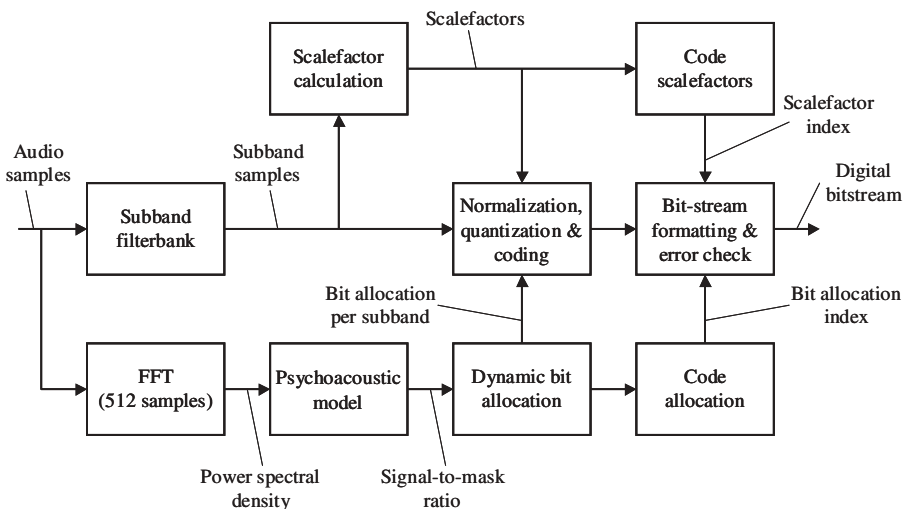


Figure 9.2 Basic block diagram of a Layer I encoder.

the individual components of the encoder are explained in more detail in the following sections.

For a Layer I encoder, the analysis filter bank converts 384 input PCM samples into a *block* of 12 sets of 32 subband samples. A *scalefactor* is then calculated for each subband and used to normalize the subband samples to a range of -1 to $+1$. The same scalefactor is used to normalize the 12 samples in each subband. In parallel to the filter bank, a psychoacoustic model is used to estimate the masking threshold associated with the current set of input samples. The masking threshold indicates the level of inaudible quantization noise that can be added to the subband samples during the quantization process. A dynamic bit allocation process then uses this masking threshold information to determine the number of bits required to transmit each subband sample. The block of normalized subband samples is then quantized using the quantizer step size that corresponds to the number of bits allocated for each subband. Then finally, the bits for the coded bit allocation, scalefactor information, and quantized subband samples are combined with header information and formatted into an audio *frame*.

9.1.1. Analysis Filterbank

The analysis filterbank used in the Layer I,II encoders is based on the efficient implementation of a cosine modulated filterbank proposed by Joseph Rothweiler in 1983. A detailed explanation of this filterbank technique is given in Section 8.5 of Chapter 8. The first 256 values of the modified prototype filter $c(k)$ are given in Table 9.1. The value of $c(257)$ is 0.03578097 and the remaining 255 values are found by time-reversing and negating the values with indexes 1 to 255 from Table 9.1.

The subband filterbank used in the Layer I,II encoders has the following minor limitations:

1. The individual frequency responses of adjacent subbands overlap, so a single frequency can affect two adjacent subband sample values.
2. The filterbank does not provide perfect reconstruction, so even if no quantization is performed there is still a very small error introduced into the output samples.
3. The equal width subbands do not correspond well to the unequal critical bandwidths of the human ear, especially at low frequencies.

9.1.2. Scalefactor Calculation

For a Layer I encoder, a *scalefactor* is calculated for each set of 12 subband samples. This scalefactor is used later in the calculation of the signal-to-mask ratio and also in the quantization and encoding of the subband samples. The scalefactor is calculated, for each subband, by finding the maximum of the absolute values of the 12 subband samples and then taking the lowest value from Table 9.2 that is greater than this maximum value.

The index in Table 9.2 is transmitted as a 6-bit binary number (MSB first) if a nonzero number of bits has been allocated to the subband in the bit allocation procedure.

Table 9.1 Values for the modified prototype filter used in the Layer I,II MPEG-1 audio coders.

	$w(8*i+j)$							
	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=0$	0.000000000	-0.000000477	-0.000000477	-0.000000477	-0.000000477	-0.000000477	-0.000000477	-0.000000954
$i=1$	-0.000000954	-0.000000954	-0.000000954	-0.000001431	-0.000001431	-0.000001907	-0.000001907	-0.000002384
$i=2$	-0.000002384	-0.000002861	-0.000003338	-0.000003338	-0.000003815	-0.000004292	-0.000004768	-0.000005245
$i=3$	-0.000006199	-0.000006676	-0.000007629	-0.000008106	-0.000009060	-0.000010014	-0.000011444	-0.000012398
$i=4$	-0.000013828	-0.000014782	-0.000016689	-0.000018120	-0.000019550	-0.000021458	-0.000023365	-0.000025272
$i=5$	-0.000027657	-0.000030041	-0.000032425	-0.000034809	-0.000037670	-0.000040531	-0.000043392	-0.000046253
$i=6$	-0.000049591	-0.000052929	-0.000055790	-0.000059605	-0.000062943	-0.000066280	-0.000070095	-0.000073433
$i=7$	-0.000076771	-0.000080585	-0.000083923	-0.000087261	-0.000090599	-0.000093460	-0.000096321	-0.000099182
$i=8$	0.000101566	0.000103951	0.000105858	0.000107288	0.000108242	0.000108719	0.000108719	0.000108242
$i=9$	0.000106812	0.000105381	0.000102520	0.000099182	0.000095367	0.000090122	0.000084400	0.000077724
$i=10$	0.000069618	0.000060558	0.000050545	0.000039577	0.000027180	0.000013828	-0.000000954	-0.000017166
$i=11$	-0.000034332	-0.000052929	-0.000072956	-0.000093937	-0.000116348	-0.000140190	-0.000165462	-0.000191212
$i=12$	-0.000218868	-0.000247478	-0.000277042	-0.000307560	-0.000339031	-0.000371456	-0.000404358	-0.000438213
$i=13$	-0.000472546	-0.000507355	-0.000542164	-0.000576973	-0.000611782	-0.000646591	-0.000680923	-0.000714302
$i=14$	-0.000747204	-0.000779152	-0.000809669	-0.000838757	-0.000866413	-0.000891685	-0.000915051	-0.000935555
$i=15$	-0.000954151	-0.000968933	-0.000980854	-0.000989437	-0.000994205	-0.000995159	-0.000991821	-0.000983715
$i=16$	0.000971317	0.000953674	0.000930786	0.000902653	0.000868797	0.000829220	0.000783920	0.000731945
$i=17$	0.000674248	0.000610352	0.000539303	0.000462532	0.000378609	0.000288486	0.000191689	0.000088215
$i=18$	-0.000021458	-0.000137329	-0.000259876	-0.000388145	-0.000522137	-0.000661850	-0.000806808	-0.000956535
$i=19$	-0.001111031	-0.001269817	-0.001432419	-0.001597881	-0.001766682	-0.001937389	-0.002110004	-0.002283096
$i=20$	-0.002457142	-0.002630711	-0.002803326	-0.002974033	-0.003141880	-0.003306866	-0.003467083	-0.003622532
$i=21$	-0.003771782	-0.003914356	-0.004048824	-0.004174709	-0.004290581	-0.004395962	-0.004489899	-0.004570484

(continued)

Table 9.1 (Continued)

	$w(8*i+j)$							
	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=22$	-0.004638195	-0.004691124	-0.004728317	-0.004748821	-0.004752159	-0.004737377	-0.004703045	-0.004649162
$i=23$	-0.004573822	-0.004477024	-0.004357815	-0.004215240	-0.004049301	-0.003858566	-0.003643036	-0.003401756
$i=24$	0.003134727	0.002841473	0.002521515	0.002174854	0.001800537	0.001399517	0.000971317	0.000515938
$i=25$	0.000033379	-0.000475883	-0.001011848	-0.001573563	-0.002161503	-0.002774239	-0.003411293	-0.004072189
$i=26$	-0.004756451	-0.005462170	-0.006189346	-0.006937027	-0.007703304	-0.008487225	-0.009287834	-0.010103703
$i=27$	-0.010933399	-0.011775017	-0.012627602	-0.013489246	-0.014358521	-0.015233517	-0.016112804	-0.016994476
$i=28$	-0.017876148	-0.018756866	-0.019634247	-0.020506859	-0.021372318	-0.022228718	-0.023074150	-0.023907185
$i=29$	-0.024725437	-0.025527000	-0.026310921	-0.027073860	-0.027815342	-0.028532982	-0.029224873	-0.029890060
$i=30$	-0.030526638	-0.031132698	-0.031706810	-0.032248020	-0.032754898	-0.033225536	-0.033659935	-0.034055710
$i=31$	-0.034412861	-0.034730434	-0.035007000	-0.035242081	-0.035435200	-0.035586357	-0.035694122	-0.035758972

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.2 Layer I,II Scalefactors.

Index	Scalefactor	Index	Scalefactor
0	2.000000000000000	32	0.00123039165029
1	1.58740105196820	33	0.00097656250000
2	1.25992104989487	34	0.00077509816991
3	1.000000000000000	35	0.00061519582514
4	0.79370052598410	36	0.00048828125000
5	0.62996052494744	37	0.00038754908495
6	0.500000000000000	38	0.00030759791257
7	0.39685026299205	39	0.00024414062500
8	0.31498026247372	40	0.00019377454248
9	0.250000000000000	41	0.00015379895629
10	0.19842513149602	42	0.00012207031250
11	0.15749013123686	43	0.00009688727124
12	0.125000000000000	44	0.00007689947814
13	0.09921256574801	45	0.00006103515625
14	0.07874506561843	46	0.00004844363562
15	0.062500000000000	47	0.00003844973907
16	0.04960628287401	48	0.00003051757813
17	0.03937253280921	49	0.00002422181781
18	0.031250000000000	50	0.00001922486954
19	0.02480314143700	51	0.00001525878906
20	0.01968626640461	52	0.00001211090890
21	0.015625000000000	53	0.00000961243477
22	0.01240157071850	54	0.00000762939453
23	0.00984313320230	55	0.00000605545445
24	0.007812500000000	56	0.00000480621738
25	0.00620078535925	57	0.00000381469727
26	0.00492156660115	58	0.00000302772723
27	0.003906250000000	59	0.00000240310869
28	0.00310039267963	60	0.00000190734863
29	0.00246078330058	61	0.00000151386361
30	0.001953125000000	62	0.00000120155435
31	0.00155019633981		

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

9.1.3. Psychoacoustic Model 1

The details of how to implement a psychoacoustic model are *nonnormative*. This means that it is not necessary to define these implementation details in order to produce a bitstream that can be decoded by a decoder that complies with the standard. However, the MPEG-1 audio standard provides an *informative* annex that specifies the implementation details of two psychoacoustic models. It is suggested that model 1

be used for Layers I and II and model 2 be used in layer III. The psychoacoustic model is used to calculate the signal-to-mask ratio (SMR) for each subband. This information is then used to select the quantizer step size for each subband that guarantees that the quantization noise introduced is inaudible.

More specifically, the psychoacoustic model defines an algorithm to estimate the masking threshold for the current input audio signal. (For a more detailed explanation of auditory masking see Chapter 7.) This masking threshold is used to determine the number of bits required to transmit the subband samples, in a particular subband, with no audible quantization noise as shown in Figure 9.3. For each subband, the minimum level of the masking threshold indicates the worst-case level of masking. The SMR is then determined using this minimum masking level and the maximum signal level in that subband. Then, because the subband samples are normalized before they are quantized, there is a well-defined relationship between the number of bits used to transmit the subband sample and the signal-to-noise ratio (SNR) introduced by quantizing to this number of bits. To guarantee no quantization noise is audible in the decoded audio signal, it is simply a matter of allocating the number of bits to each subband that guarantees that the SNR is greater than the SMR as shown in Figure 9.3.

In psychoacoustic model 1 the SNR is calculated using the following nine steps:

1. Calculate the FFT of the input audio samples.
2. Determine the maximum sound pressure level in each subband.
3. Determine the threshold in quiet.
4. Find the tonal and nontonal components of the input audio signal.
5. Decimate the maskers to obtain only the relevant maskers.
6. Calculate the individual masking thresholds for each relevant masker.

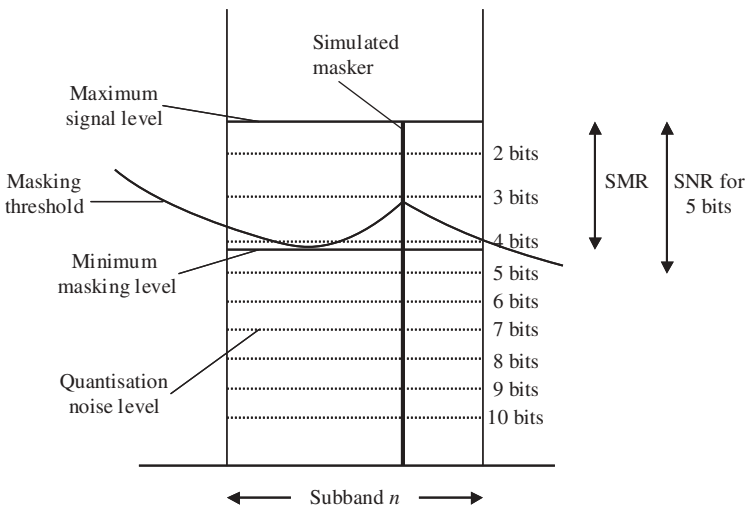


Figure 9.3 The masking threshold produced by the psychoacoustic model is used to determine the SMR for each subband.

7. Determine the global masking threshold.
8. Determine the minimum masking threshold in each subband.
9. Calculate the SMR ratio.

These nine steps are explained in more detail in the following sections.

9.1.3.1. Calculating the FFT of the Input Audio Samples

An estimate of the power spectral density of the audio samples is calculated using a 512-point FFT for Layer I or a 1024-point FFT for Layer II. In each case, the input samples are first multiplied by a Hann window function given by

$$w(n) = \sqrt{\frac{8}{12}} \left[1 - \cos\left(\frac{2n\pi}{N}\right) \right] \quad \text{for } n = 0, \dots, N-1 \quad (9.1)$$

where N is the number of points used to calculate the FFT. Because of the delay through the analysis filter, for the output of the FFT to coincide with the output sub-band samples of the analysis filter, a delay must be introduced at the input to the FFT. For Layer I, the input samples of the FFT should be delayed by 320 samples, and for Layer II the delay should be 192 samples.

Figure 9.4(a) shows the relative positions of the prototype analysis filter at the beginning and end of a Layer I frame, and Figure 9.4(b) shows the position of the Hann window used in the power spectral density calculation. Figure 9.4 shows that

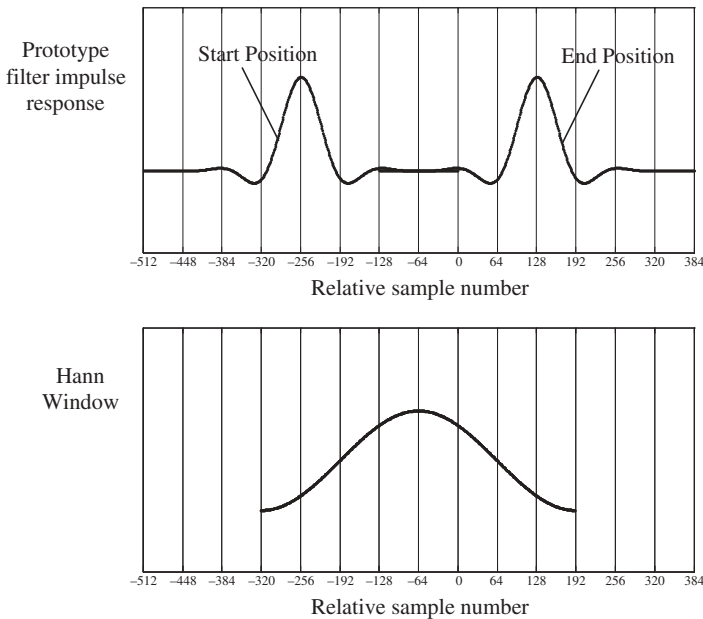


Figure 9.4 The relative position of the analysis filter and the Hann window function used in the FFT calculation.

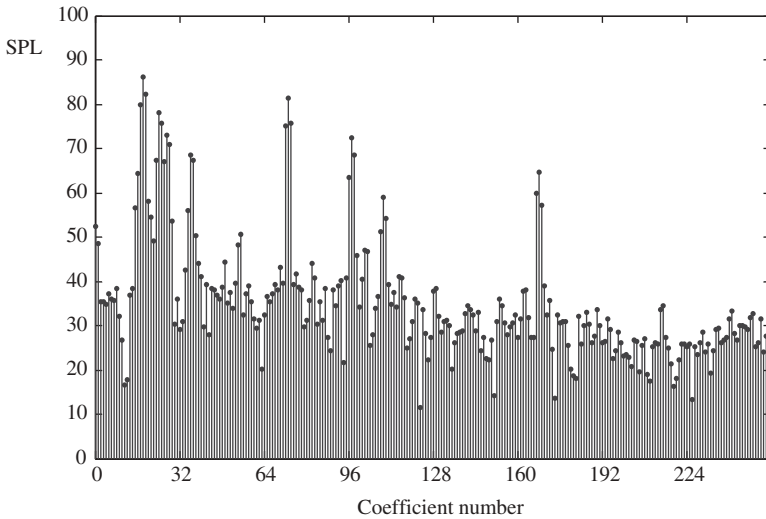


Figure 9.5 The power spectral density of a typical audio signal for a Layer 1 frame.

a delay of 320 samples is required so that the power spectral density calculation is operating on the same set of input samples as the analysis filter. Note that the Hann window does not extend to all input samples used in the analysis filter bank for the current frame. This disparity has a negligible effect because the samples lying outside the extent of the Hann window do not contribute significantly to the values of the output subband samples.

The power spectral density of the delayed input samples $x(n)$ is then given by

$$X(k) = 10 \log_{10} \left(\left| \frac{1}{N} \sum_{n=0}^{N-1} w(n) x(n) e^{-j(2nk\pi/N)} \right|^2 \right) \quad \text{for } k = 0, \dots, N/2 - 1 \quad (9.2)$$

To account for variations in the dynamic range of the input sample values, the power spectral density values should be normalized so that the maximum amplitude possible corresponds to a sound pressure level of 96 dB. Therefore, if the input samples represent a pure sinusoid with amplitude 2, the power spectral density values should be scaled so that the maximum value of $X(k)$ is 96 dB. Figure 9.5 shows an example of the power spectral density of a typical audio signal for a Layer I frame.

9.1.3.2. Determining the Maximum Sound Pressure Level in Each Subband

The power spectral density and the scalefactors for the frame are then used to determine the maximum sound pressure level in each subband. For Layer I, there

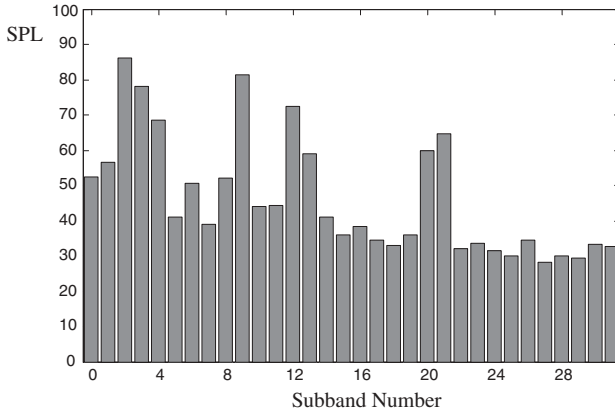


Figure 9.6 The maximum sound pressure level for each subband of the audio signal in Figure 9.5.

are eight spectral lines per subband and for Layer II there are 16 spectral lines per subband. The maximum sound pressure level of subband n is taken as the maximum of the spectral lines in the subband and the scalefactor for the subband converted to decibels. So the maximum sound pressure level, L_{sb} , of subband n is given by

$$L_{sb}(n) = \max_{k \text{ in subband } n} [X(k), 20 \log_{10}(32768 \times \text{scf}_{\max}(n)) - 10] \quad (9.3)$$

In Layer I, the expression $\text{scf}_{\max}(n)$ is simply the scalefactor for subband n , but in Layer II it denotes the maximum of the three scalefactors for subband n . Figure 9.6 shows the maximum sound pressure level for each subband of the audio signal from the previous example.

The following MATLAB code defines the algorithm used to determine the maximum sound pressure level in each subband. The array X contains the sound pressure level values for the FFT coefficients and scf_max contains the maximum scalefactor for each subband. The elements of the array L_{sb} are set to the maximum sound pressure level in each subband.

```
% step through the FFT coefficients corresponding to each subband
n = 1;
for k = 1:8:256
    % convert the maximum scalefactor for the nth subband to decibels
    scf_max_level(n) = 20*log10(32768*scf_max(n))-10;
    % find the maximum of the scalefactor level and the FFT coefficients
    % corresponding to subband n
    max_level = -inf;
    for m = 0:subband_width-1
        max_level = max([max_level X(k+m) scf_max_level(n)]);
    end
end
```

```

% set the maximum sound pressure level for subband n to this
% maximum level
Lsb(n) = max_level;
n = n+1;
end

```

9.1.3.3. Determining the Threshold in Quiet

The absolute threshold or the *threshold in quiet*, LT_q , is given in Tables A.1 and A.2 in Appendix for Layers I and II, respectively. Note that the values in these tables are given for a subset (indexed by i) of the original $N/2$ frequency values (indexed by k).

For Layer I, no subsampling is used for the first six subbands. For the next six subbands, every second frequency value is considered. Then for input sampling rates of 44.1 and 48 kHz, every fourth frequency value is considered up to a maximum of 20 kHz and for 32 kHz, every fourth frequency value is considered up to a maximum of 15 kHz.

For Layer II, no subsampling is used for the first three subbands and for the next three subbands, every second frequency value is considered. For the next six subbands, every fourth frequency value is considered. Then for input sampling rates of 44.1 and 48 kHz, every eighth frequency value is considered up to a maximum of 20 kHz and for 32 kHz, every eighth frequency value is considered up to a maximum of 15 kHz.

EXAMPLE 9.1

For a Layer I frame with an input sample rate of 44.1 kHz, the following equation describes the relationship between the index i of the subset of frequency values and the index k of the original frequency values:

$$i = \begin{cases} k, & \text{for } k = 1, 2, \dots, 48 \\ \frac{k}{2} + 24, & \text{for } k = 50, 52, \dots, 96 \\ \frac{k}{4} + 48, & \text{for } k = 100, 104, \dots, 232 \end{cases}$$

■

9.1.3.4. Finding the Tonal and Nontonal Masking Components

The masking effect is different for tonal (sinusoidal) and nontonal maskers so it is necessary to determine the position of both tonal and nontonal masking components from the power spectral density of the input signal. The process for determining the location of these masking components is explained below.

Locating Local Maxima Because a tonal masker produces a large narrow spike in the power spectral density of the signal, the first step in this process is to determine the position of the local maxima.

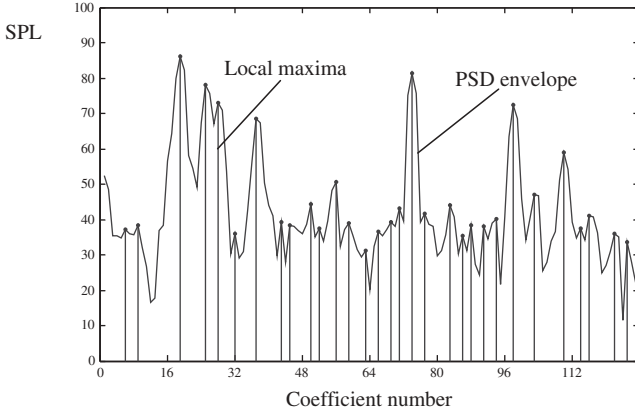


Figure 9.7 The envelope of the power spectral density and the local maxima.

A spectral line in $X(k)$ is considered to be a local maximum if the following condition holds:

$$X(k) > X(k-1) \quad \text{and} \quad X(k) > X(k+1) \quad (9.4)$$

Figure 9.7 shows the envelope for the first half of the power spectral density from the previous example with those components identified as local maxima using the above criteria.

The following MATLAB code defines the algorithm used to locate the local maxima. The elements of the array `is_loc_max` are set to 1 when an FFT coefficient is greater than its neighbors above and below in frequency.

```
is_loc_max = zeros(1,256);
for k = 2:255
    if ((X(k) > X(k-1)) && (X(k) > X(k+1)))
        is_loc_max(k) = 1;
    else
        is_loc_max(k) = 0;
    end
end
```

Locating Tonal Maskers Not every local maximum represents a tonal masker. The next step requires the location of maxima that are significantly greater in amplitude than their neighboring spectral components. Because the human auditory system has a better frequency resolution at lower frequencies, tonal components at lower frequencies can be closer together than at higher frequencies and still be heard as two distinct tones. For this reason, a larger number of neighboring spectral components are considered as the frequency of the local maxima increases. So a local maximum is considered to be a tonal masker if the following equation holds

$$X(k) - X(k-j) \geq 7 \text{ dB} \quad (9.5)$$

where j is determined according to the value of k and is given by the following equations.

Layer I:

$$j = \begin{cases} -2, 2, & \text{for } 2 < k < 63 \\ -3, -2, 2, 3, & \text{for } 64 \leq k < 127 \\ -6, \dots, -2, 2, \dots, 6, & \text{for } 128 \leq k < 250 \end{cases}$$

Layer II:

$$j = \begin{cases} -2, 2, & \text{for } 2 < k < 63 \\ -3, -2, 2, 3, & \text{for } 64 \leq k < 127 \\ -6, \dots, -2, 2, \dots, 6, & \text{for } 128 \leq k < 255 \\ -12, \dots, -2, 2, \dots, 12, & \text{for } 256 \leq k \leq 500 \end{cases}$$

If a local maximum at frequency index k satisfies this equation then the sound pressure level of the tonal masker is given by

$$X_m(k) = 10 \log_{10} \left(10^{\frac{X(k-1)}{10}} + 10^{\frac{X(k)}{10}} + 10^{\frac{X(k+1)}{10}} \right) \text{ dB} \quad (9.6)$$

Figure 9.8 shows the envelope for the first half of the power spectral density from the previous example together with those components identified as tonal maskers using the above criteria.

So the tonal maskers are not considered when calculating the sound pressure level of the nontonal maskers, all spectral components within the neighborhood of a tonal masker are set to $-\infty$ dB. The following MATLAB code defines the algorithm used to locate the tonal maskers. The elements of the array `is_tonal_masker` are set to 1 when an FFT coefficient is found to be a tonal masker. The elements of the array

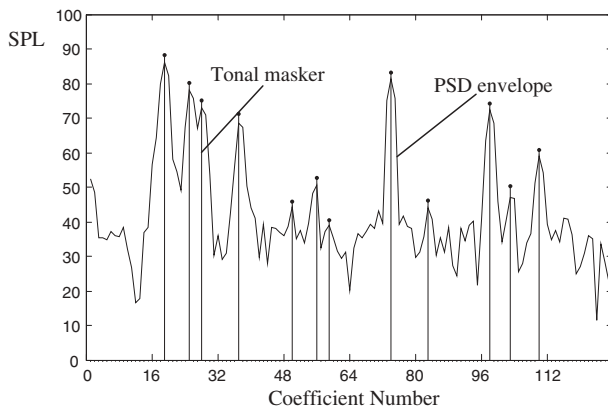


Figure 9.8 The envelope of the power spectral density and the tonal maskers.

Xtm are set to the simulated SPL value for a single tone masker at the location of the masker and minus infinity elsewhere.

```

is_tonal_masker = zeros(1,N);
Xtm = -inf*ones(1,256);

% step through the FFT coefficients one at a time
for k = 3:250
    % if the coefficient has previously been identified as a local maxima
    % check to determine if it is at a sufficient higher level compared to
    % its neighbours
    if (is_loc_max(k) == 1)
        % the frequency of the coefficient is used to determine the number
        % of coefficients to consider as neighbours
        if ((k >= 3) && (k < 64))
            neighbour_coeff = [-2 2];
            num_coeff = 2;
        elseif ((k >= 64) && (k < 128))
            neighbour_coeff = [-3 -2 2 3];
            num_coeff = 4;
        else
            neighbour_coeff = [-6 -5 -4 -3 -2 2 3 4 5 6];
            num_coeff = 10;
        end
        % if one of the coefficient's neighbours is within 7 dB of the
        % coefficient then the tonal masker criterion is not satisfied
        satisfied = 1;
        for j = 1:num_coeff
            if (X(k) - X(k+neighbour_coeff(j)) < 7)
                satisfied = 0;
            end
        end
        % if the coefficient satisfies the criterion for a tonal masker
        if (satisfied)
            % calculate the level for a simulated single tone tonal masker
            masker_sum = 10^(X(k-1)/10) + 10^(X(k)/10) + 10^(X(k+1)/10);
            Xtm(k) = 10*log10(masker_sum);
            % set a boolean variable to locate the tonal maskers
            is_tonal_masker(k) = 1;
            % set coefficients in neighbourhood of masker to minus infinity
            % for the calculation of non-tonal maskers
            for j = 1:num_coeff
                X(k+neighbour_coeff(j)) = -inf;
            end
            X(k-1:k+1) = [-inf -inf -inf];
        end
    end
end
end

```

Locating Nontonal Maskers The SPL of the nontonal masking components are calculated from the spectral lines remaining after the components associated with the tonal maskers have been removed. The input frequency range is first divided up into a series of critical bands. Then one nontonal masking component is calculated for each critical band. The SPL of a nontonal masking component is calculated for each critical band by taking the sum of the power of the spectral components in the critical band (after the tonal components have been removed) and is given by

$$X_{nm}(k_{cb}) = 10 \log_{10} \left(\sum_{k=k_l}^{k_u} 10^{(X(k)/10)} \right) \quad (9.7)$$

where k_l is the FFT index corresponding to the lowest frequency in the critical band and k_u is the FFT index corresponding to the highest frequency in the critical band. The location of the nontonal component for each critical band, k_{cb} , is taken as the index that is closest to the geometric mean of the indices of the critical band.

The following MATLAB code defines the algorithm used to locate the tonal maskers. The arrays `lower`, `geom_mean`, and `upper` contain the FFT index of the lower, geometric mean, and upper frequencies in each critical band and are given in Tables A.3 and A.4 in Appendix. The elements of the array `is_non_tonal_masker` are set to 1 for the index that is closest to the geometric mean of the indices of the critical band. The elements of the array `Xnm` are set to the simulated SPL value for a single tone masker at the location of the masker and minus infinity elsewhere.

```
is_non_tonal_masker = zeros(1,256);
Xnm = -inf*ones(1,256);
% step through each critical band
for crit_band = 1:num_critical_band
    % find the linear sum of the power of the SPL values in the critical
    % band after the tonal maskers have been removed
    linear_sum = 0;
    for k = lower(crit_band):upper(crit_band)
        linear_sum = linear_sum+10^(X(k)/10);
    end
    % set a boolean variable to locate the non-tonal maskers at the
    % coefficient index that is closest to the geometric mean of the indexes
    % in the critical band
    is_non_tonal_masker(geom_mean(crit_band)) = 1;
    % set the level for a simulated single tone tonal masker
    Xnm(geom_mean(crit_band)) = 10*log10(linear_sum);
end
end
```

Figure 9.9 shows the envelope for the first half of the power spectral density from the previous example (after the components in the neighborhood of the tonal

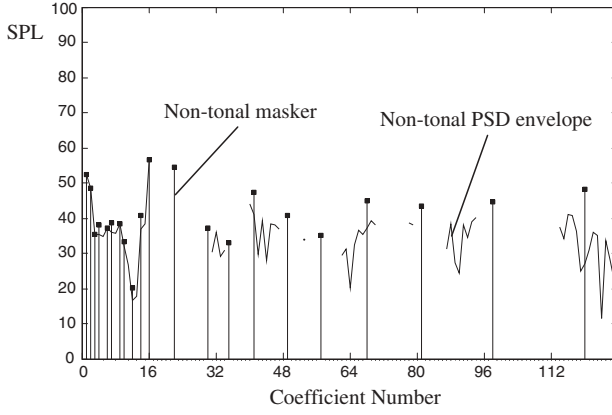


Figure 9.9 The envelope of the power spectral density after the components contributing to tonal masking have been removed and the nontonal maskers.

maskers have been removed) and the nontonal maskers for each critical band calculated using the above equations.

9.1.3.5. Decimating the Tonal and Nontonal Masking Components

Not all maskers from the previous step are used to calculate the global masking threshold. In this step, maskers can be removed if they do not meet certain requirements. Tonal or nontonal masking components are removed if their SPL is less than the absolute threshold, that is, tonal and nontonal masking components must satisfy the following conditions.

Tonal masking components:

$$X_m(k) \geq LT_q(k) \quad (9.8)$$

Nontonal masking components:

$$X_{nm}(k) \geq LT_q(k) \quad (9.9)$$

The following MATLAB code defines the algorithm used to decimate the tonal and nontonal maskers that are below the threshold-in-quiet. The array $LT_q(k)$ contains the absolute threshold values at the frequency indicated by k . These absolute threshold values can be calculated for each input sampling rate from the values in Tables A.1 and A.2 of the Appendix.

```
% step through the FFT coefficients
for k = 1:256
    if (is_tonal_masker(k))
        % if the level of a tonal masker is below the threshold-in-quiet
        % then remove it
```

```

    if (Xtm(k) < LTq(k))
        is_tonal_masker(k) = 0;
        Xtm(k) = -inf;
    end
elseif (is_non_tonal_masker(k))
    % if the level of a non-tonal masker is below the threshold-in-quiet
    % then remove it
    if (Xnm(k) < LTq(k))
        is_non_tonal_masker(k) = 0;
        Xnm(k) = -inf;
    end
end
end
end
end

```

In addition, if two tonal masking components have critical band rates that are within 0.5 Bark of each other, the component with the lowest power is removed. The following MATLAB code defines the algorithm used to decimate the tonal maskers that are within 0.5 Bark of a larger tonal masker. The array `band_rate(k)` contains the critical band rate that corresponds to the frequency of the index `k`. The frequency to band rate conversion can be calculated using Equation (7.2) in Chapter 7.

```

% In each iteration find the smallest difference in critical band rate
% between two tonal maskers. If this difference is smaller than 0.5 then
% remove the tonal masker with the lowest power. Stop when no two tonal
% maskers are less than 0.5 bark apart.
% set the minimum difference in band rate to zero to start the while loop
min_dz = 0;
while min_dz < 0.5
    min_dz = inf;
    % step through the FFT coefficients
    for k = 1:256
        if is_tonal_masker(k)
            % if the coefficient is a tonal masker find its critical band
            % rate
            z1 = band_rate(k);
            % now step through the remaining coefficients
            for j = 1:256-k
                if is_tonal_masker(k+j)
                    % if a second tonal masker is found find its critical
                    % band rate
                    z2 = band_rate(k+j);
                    % find the difference in critical band rate for these
                    % two tonal maskers
                    dz = z2-z1;
                    % check to see if this is the minimum difference

```

```

% between two coefficients
if dz < min_dz
    % if this is the minimum difference update the
    % minimum and remember the index of the coefficient
    % with the lowest power
    min_dz = dz;
    if Xtm(k) < Xtm(k+j)
        remove = k;
    else
        remove = k+j;
    end
end
end
end
end
end
% if the minimum difference is less than 0.5 bark remove the
% coefficient with the lowest power
if min_dz < 0.5
    Xtm(remove) = -inf;
    is_tonal_masker(remove) = 0;
end
end
end

```

Figure 9.10 shows the envelope for the first half of the power spectral density from the previous example together with the tonal and nontonal maskers that are used to calculate the individual masking thresholds in the next step.

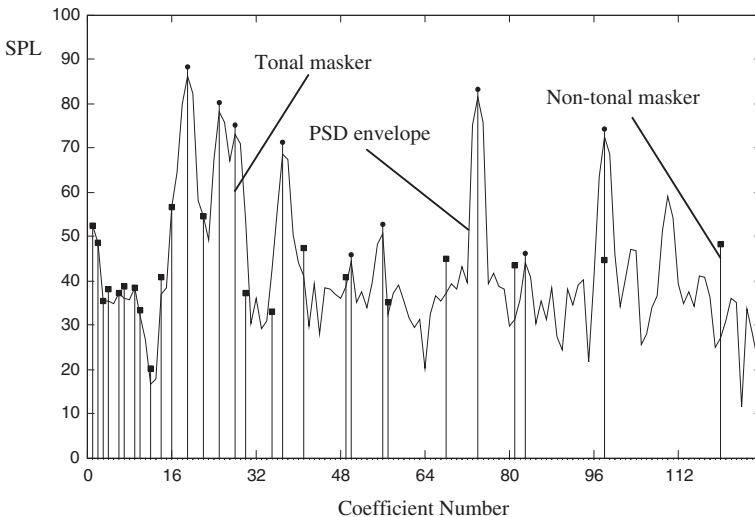


Figure 9.10 The envelope of the power spectral density and the tonal and nontonal maskers.

9.1.3.6. Calculating the Individual Masking Thresholds

The individual masking thresholds for the i th masking components at frequencies with index j are given by the following equations.

For tonal maskers:

$$LT_{im}(i, j) = X_{im}(k) + av_{im}(k) + vf(k, j) \quad (9.10)$$

For non-tonal maskers:

$$LT_{nm}(i, j) = X_{nm}(k) + av_{nm}(k) + vf(k, j) \quad (9.11)$$

In these equations, k is the frequency index of the i th masking component, the terms $av_{im}(k)$ and $av_{nm}(k)$ denote the *masking index* for tonal and nontonal maskers, respectively, and the term $vf(k, j)$ denotes the value of the *masking function* of the i th masking component at frequency index j . The masking index differs for tonal and nontonal maskers and is given by the following equations.

For tonal maskers:

$$av_{im}(k) = -0.275 z(k) - 6.025 \quad (9.12)$$

For nontonal maskers:

$$av_{nm}(k) = -0.175 z(k) - 2.025 \quad (9.13)$$

In these equations, the term $z(k)$ denotes the critical band rate of the masker and can be found using the equation:

$$z(k) = \frac{28f(k)}{f(k) + 2200} - 0.5 \text{ Bark} \quad (9.14)$$

where $f(k)$ is the frequency value corresponding to the index k .

The masking index sets a constant offset for the masking threshold of a masker. This offset is lower and also decreases more rapidly with critical band rate for tonal maskers than for nontonal maskers. The masking function defines the slope of the masking threshold for frequencies above and below the critical band rate of the masker. The masking function defines four different slopes for the masking threshold and is given by

$$vf(k, j) = \begin{cases} 17(dz + 1) - (0.4X(k) + 6), & \text{for } -3 \leq dz < -1 \\ (0.4X(k) + 6)dz, & \text{for } -1 \leq dz < 0 \\ -17dz, & \text{for } 0 \leq dz < 1 \\ -(dz - 1)(17 - 0.15X(k)) - 17, & \text{for } 1 \leq dz < 8 \end{cases} \quad (9.15)$$

where dz is the distance in Bark from the masker and is given by $dz = z(j) - z(k)$. The individual masking threshold for each masker is set to $-\infty$ dB for critical

band rates less than 3 Bark below and greater than 8 Bark above the critical band rate of the masker. The following MATLAB code defines the algorithm used to calculate the individual masking thresholds. The elements of the array $LTm(i,j)$ are set to the masking threshold values for the i th masking component at frequency index j .

```

LTm = -inf*ones(1,256);
i = 0;
% step through each FFT coefficient
for k = 1:256
    % if the coefficient is a masker
    if (is_tonal_masker(k) &Vert;&Vert; is_non_tonal_masker(k))
        % increment the index for the masker
        i = i+1;
        % find the critical band rate corresponding to the frequency of the
        % masker
        z = band_rate(k);
        % for this masker step through each frequency index
        for j = 1:256
            % find the difference in critical band rate between this
            % frequency index to the critical band rate of the masker
            dz = band_rate(j) - z;
            % calculate the individual masking threshold for this tonal
            % masker
            if (is_tonal_masker(k))
                avtm = -1.525 - 0.275*z - 4.5;
                vf = masking_function(dz,Xtm(k));
                LTm(i,j) = Xtm(k) + avtm + vf;
            end
            % calculate the individual masking threshold for this non-tonal
            % masker
            if (is_non_tonal_masker(k))
                avnm = -1.525 - 0.175*z - 0.5;
                vf = masking_function(dz,Xnm(k));
                LTm(i,j) = Xnm(k) + avnm + vf;
            end
        end
    end
end
end
end

function vf = masking_function(dz,X)
% calculate the masking function for the critical band rate difference dz
% and masker SPL value X
if (dz >= -3 && dz < -1)
    vf = 17*(dz + 1) - 0.4*X - 6;

```

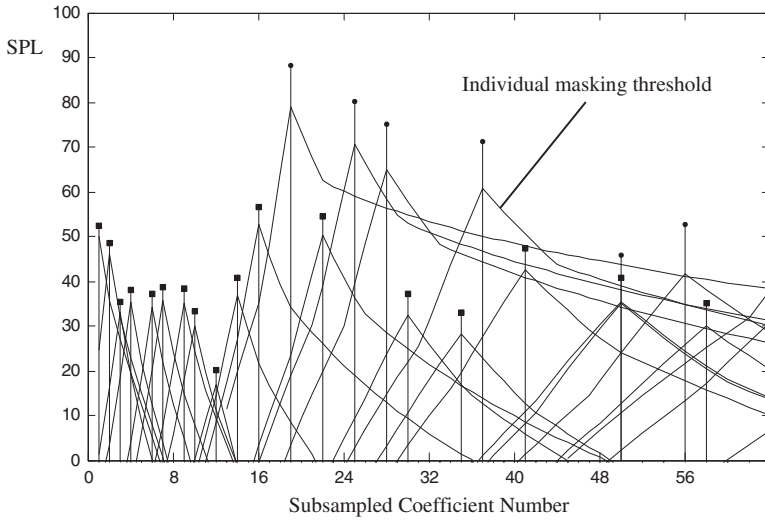


Figure 9.11 Individual masking thresholds for a subset of the tonal and nontonal maskers

```

elseif (dz >= -1 && dz < 0)
    vf = (0.4*X + 6)*dz;
elseif (dz >= 0 && dz < 1)
    vf = -17*dz;
elseif (dz >= 1 && dz < 8)
    vf = -(dz - 1)*(17 - 0.15*X) - 17;
else
    vf = -inf;
end

```

Figure 9.11 shows the individual masking thresholds for a subset of the tonal and nontonal maskers from the previous example.

9.1.3.7. Calculating the Global Masking Threshold

The global masking threshold is calculated for each frequency value by taking the sum of the individual masking thresholds, and the absolute threshold, and is given by

$$LT_g(k) = 10 \log_{10} \left(10^{\frac{LT_a(k)}{10}} + \sum_{i=1}^{N_m} 10^{\frac{LT_m(i,k)}{10}} \right) \quad (9.16)$$

where N_m is the number maskers.

Figure 9.12 shows the global masking threshold calculated from the individual masking thresholds from the previous example.

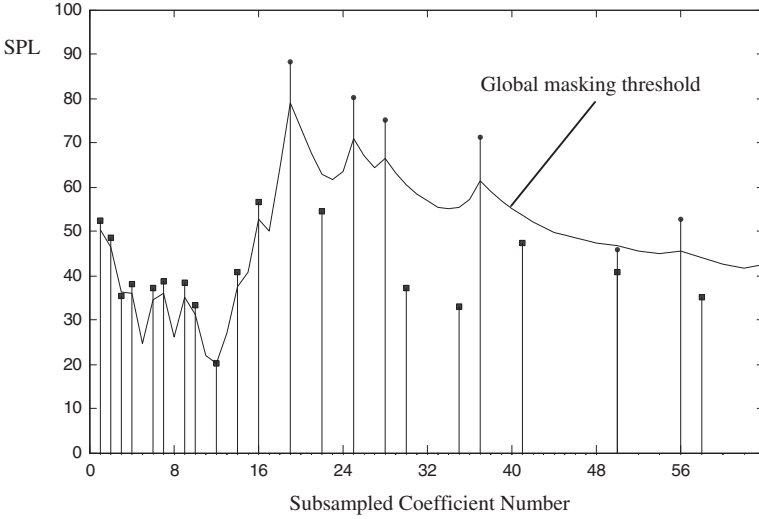


Figure 9.12 The global masking threshold for a subset of the tonal and nontonal maskers.

9.1.3.8. Calculating the Minimum Masking Threshold

The minimum masking threshold is calculated for each subband by finding the minimum of the global masking threshold over the subsampled frequency values contained in the subband and is given by

$$LT_{\min}(n) = \min_{k \text{ in subband } n} (LT_g(k)) \quad (9.17)$$

Figure 9.13 shows the minimum masking threshold for each subband calculated from the global masking threshold of the previous example. The dots on the plot of the global threshold indicate the frequency values at which the global threshold was calculated.

9.1.3.9. Calculating the Signal-to-Mask Ratio

The signal-to-mask ratio is calculated by subtracting the minimum masking threshold from the sound pressure level for each subband and is given by

$$SMR(n) = L_{sb}(n) - LT_{\min}(n) \quad (9.18)$$

Figure 9.14 shows the signal-to-mask ratio for each subband calculated using the above equation.

9.1.4. Dynamic Bit Allocation

The total number of bits available for each audio frame, cb , is determined from the number of 32-bit *slots* that are to be transmitted for that frame (see Section 9.1.7). Not all of the bits in each frame are available to transmit subband sample

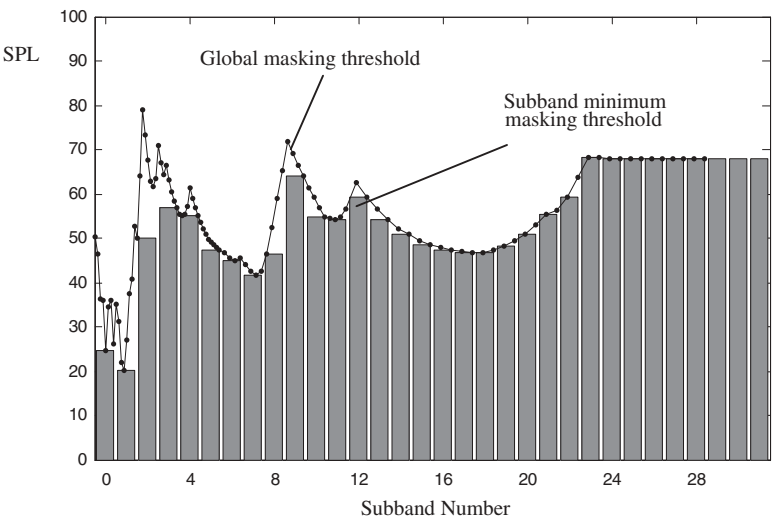


Figure 9.13 The minimum masking threshold for each subband and the global masking threshold.

and scalefactor information. The available number of bits can be determined by subtracting from the total number of bits available, *cb*, the number of bits required to transmit the header, *bhdr* (32 bits), the CRC check word if used, *bcrc*, (16 bits), the bit allocation for each subband, *bbal*, and any ancillary data, *banc* (see Section 9.4).

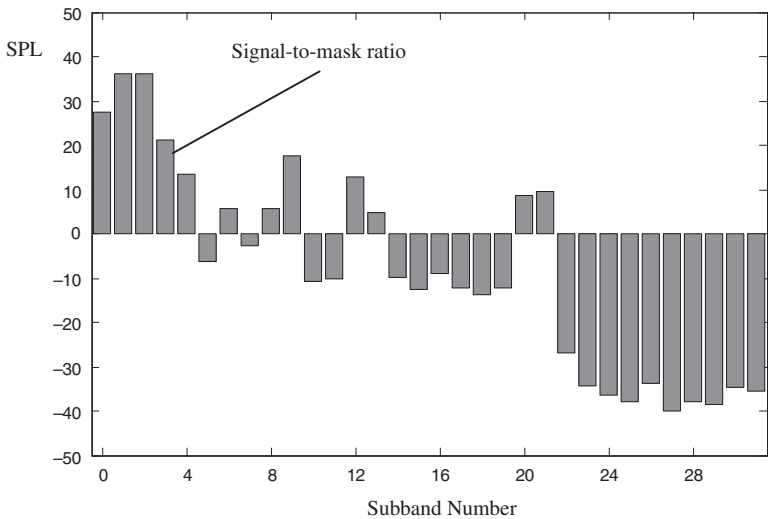


Figure 9.14 The signal-to-mask ratio for each subband.

So the number of bits available for subband sample and scalefactor information, adb , is

$$adb = cb - (bhdr + bcrc + bbal + banc) \quad (9.19)$$

The available bits are then allocated to the subbands using the principle of minimizing the total noise-to-mask ratio of the frame. The number of bits allocated to one sample can range from 0 to 15 bits with the exception of 1 bit per sample, which is not allowed.

The procedure to find the number of bits allocated to each subband begins by finding the *mask-to-noise-ratio* (MNR) for each subband. The MNR is calculated by subtracting the signal-to-mask ratio from the signal-to-noise ratio for each subband and is given by

$$\text{MNR} = \text{SNR} - \text{SMR} \quad (9.20)$$

The signal-to-mask ratio is the output of the psychoacoustic model and the signal-to-noise ratio depends on the number of bits allocated to the subband. The relationship between SNR and bits allocated to a subband is given in Table 9.3.

The allocation of bits to each subband is an iterative process that begins by allocating zero bits for the scalefactors and subband samples of each subband. The iterative process then continues using the following steps:

1. Use the number of bits allocated to each subband to determine the SNR for each subband and then calculate the MNR for each subband.

Table 9.3 SNR per number of bits allocated to each subband.

No. of bits	SNR (dB)	<i>A</i>	<i>B</i>
0	0.00		
2	7.00	0.750000000	−0.250000000
3	16.00	0.875000000	−0.125000000
4	25.28	0.937500000	−0.062500000
5	31.59	0.968750000	−0.031250000
6	37.75	0.984375000	−0.015625000
7	43.84	0.992187500	−0.007812500
8	49.89	0.996093750	−0.003906250
9	55.93	0.998046875	−0.001953125
10	61.96	0.999023438	−0.000976563
11	67.98	0.999511719	−0.000488281
12	74.01	0.999755859	−0.000244141
13	80.03	0.999877930	−0.000122070
14	86.05	0.999938965	−0.000061035
15	92.01	0.999969482	−0.000030518

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

2. Determine the subband that has the minimum MNR.
3. The number of bits allocated to this subband is then increased to the next highest allowable number of bits.
4. The number of bits required to transmit the samples, *bspl*, is updated to the number of bits required in step 2. In addition, if *bspl* has increased from zero to two then the number of bits required to transmit the scalefactor for this subband, *bscf* is increased from zero to six bits.
5. Recalculate the number of bits available for subband sample and scalefactor information, *adb*, using the following equation:

$$adb = cb - (bhdr + bcrc + bbal + bscf + bspl + banc) \quad (9.21)$$

6. If *adb* is greater than any possible increase of *bscf* + *bspl* the iterative procedure continues from step 1.

9.1.5. Coding of Bit Allocation

The bit allocation information for subband **sb** of channel **ch** is transmitted using the 4 bit code word **allocation[ch][sb]**. The number of bits per sample that corresponds to each value of the **allocation[ch][sb]** code word is shown in Table 9.4.

Table 9.4 Bits per sample for each value of the allocation index.

allocation[ch][sb]	Bits per sample
“0000”	0
“0001”	2
“0010”	3
“0011”	4
“0100”	5
“0101”	6
“0110”	7
“0111”	8
“1000”	9
“1001”	10
“1010”	11
“1011”	12
“1100”	13
“1101”	14
“1110”	15
“1111”	forbidden

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

9.1.6. Quantization and Coding of Subband Samples

The subband samples are coded using a linear quantizer with a symmetric zero representation. This type of quantizer guarantees that small values close to zero are quantized to zero. The quantization process is defined by the following steps:

1. Normalize the subband sample by dividing its value by the scalefactor to obtain the normalized sample X .
2. Calculate the quantized value Q using the following formula:

$$Q = AX + B$$

where the values for the quantization coefficients A and B depend on the number of bits allocated to the subband and are given in Table 9.3.

3. Find Q_N by taking the N most significant bits of Q where N is the number of bits allocated in the previous bit allocation process and Q is stored as a two's complement binary fraction.
4. Invert the most significant bit of Q_N in order to avoid the code word containing all 1s.

EXAMPLE 9.2

To quantize a subband sample using three bits, the first step is to normalize the subband value by the scalefactor to give a value for X . Since the scalefactor must always be larger than the maximum sample value in each subband, the possible range of values for X is defined by $-1 < X < 1$. The value of X is then used to calculate a value for Q using the equation

$$Q = 0.875 X - 0.125$$

and the possible range of values for Q is then defined by $-1 < Q < 0.75$. If Q is then stored as a two's complement binary fraction and Q_3 is taken as the first three bits of this binary fraction, there are only seven different possible values for Q_3 . Table 9.5 shows the range of values for X and Q that correspond to each of these seven values of Q_3 . The last column of Table 9.5 shows the actual code word that is transmitted after the most significant bit of Q_3 has been inverted.

Table 9.5 Ranges and code words for a 3-bit quantizer.

Range for X	Range for Q	Q_3	Code word
$-1 < X < -\frac{5}{7}$	$-1 < Q < -0.75$	100	000
$-\frac{5}{7} \leq X < -\frac{3}{7}$	$-0.75 \leq Q < -0.5$	101	001
$-\frac{3}{7} \leq X < -\frac{1}{7}$	$-0.5 \leq Q < -0.25$	110	010
$-\frac{1}{7} \leq X < \frac{1}{7}$	$-0.25 \leq Q < 0.0$	111	011
$\frac{1}{7} \leq X < \frac{3}{7}$	$0.0 \leq Q < 0.25$	000	100
$\frac{3}{7} \leq X < \frac{5}{7}$	$0.25 \leq Q < 0.5$	001	101
$\frac{5}{7} \leq X < 1$	$0.5 \leq Q < 0.75$	010	110

There is a provision in the MPEG-1 Audio standard for the inclusion of a variable amount of ancillary data provided by the user. The number of ancillary data bits is included in the calculation of the constant number of bits for each frame and reduces the number of bits available for transmitting audio data. Hence a large amount of ancillary data may result in a significant degradation of audio quality if the output bit rate is not increased to compensate for the inclusion of this extra data.

Care should also be taken that a group of bits in the ancillary data does not match the syncword that occurs at the beginning of each frame (see Section 9.4.3). If care is not taken to avoid such a match, recovering synchronization immediately after an error event may be made more difficult.

9.1.7. Formatting

The output of the audio encoder is transmitted in frames. Each frame contains the information required to decode 384 samples of the original audio signal. The Layer I and II encoders produce an output bit stream with a constant bitrate. The number of bits available to code a single frame is determined using this constant output bit rate.

Each frame must consist of an integer number of *slots*. In Layer I, a slot is defined as 32 bits and the number of slots in a frame, N_s , is calculated using the following formula:

$$N_s = \text{int} \left[\frac{384R/F_s}{32} \right] \quad (9.22)$$

where R is the output bit rate and F_s is the input sampling frequency. The length of a frame is forced to be an integer number of slots so that every frame must start on a slot boundary. This alignment of a frame to a slot boundary is done to increase the speed of resynchronization after an error event has occurred. For some combinations of bit rate and input sampling frequency, if the number of slots is kept constant at N_s , the actual output bit rate gradually falls below the desired output bit rate. This means that, in practice, the number of slots in each frame is allowed to vary between N_s and $N_s + 1$ to maintain the desired output bit rate.

EXAMPLE 9.3

For a Layer I coder with an input sampling rate of 44.1 kHz and an output bit-rate of 192 kbit/s the number of slots per frame is given by

$$\begin{aligned} N_s &= \text{int} \left[\frac{384 \text{ (samples/frame)} \times 192000 \text{ (bits/second)} / 44100 \text{ (samples/second)}}{32 \text{ (bits/slot)}} \right] \\ &= 52 \text{ (slots/frame)} \end{aligned}$$

Then, if each frame is transmitted using 52 slots, after 100 frames the desired number of bits is

$$\frac{100 \times 384 \times 192,000}{44100} = 167184 \quad (\text{bits})$$

but the actual number of bits produced by the decoder is

$$100 \times 52 \times 32 = 166,400 \text{ (bits)}$$

The actual number of output bits is 784 bits (or approximately 24 slots) less than the desired number of output bits. To compensate for this discrepancy, the encoder would need to transmit approximately every fourth frame using 53 slots instead of 52. ■

If a frame is coded using an extra slot, the value of the flag **padding_bit** is set to '1' in the header of the frame. The following MATLAB code defines the algorithm that can be used to determine if padding is necessary.

```
% for the first frame don't use an extra slot and set the variable rest to
% zero
if (frame == 0)
    rest = 0;
    padding_bit = 0;
else
    % for every other frame calculate the discrepancy between the desired
    % number of bits and the actual number of bits for a single frame
    if (layer==1)
        dif = mod(384*bitrate/sampling_frequency,32);
        bits_per_slot = 32;
    else
        dif = mod(1152*bitrate/sampling_frequency,8);
        bits_per_slot = 8;
    end
    % the variable rest is used to keep track of the difference between the
    % desired and actual number of bits
    rest = rest-dif;
    % when the value of rest is negative an extra slot is required to keep
    % the difference between the desired and actual number of bits used to
    % within the size of one slot
    if (rest < 0)
        % if an extra slot is required set the value of padding_bit to 1
        % and add the number of bits in a slot to the value of rest
        padding_bit = 1;
        rest = rest + bits_per_slot;
    else
        padding_bit = 0
    end
end
```

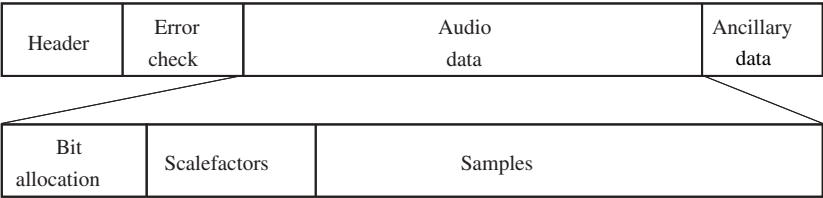


Figure 9.15 Format for a Layer I audio frame.

Figure 9.15 shows the format of a Layer I frame. The bit-stream syntax required for each audio frame is described later in Section 9.4.

9.2. LAYER II ENCODER

The major components that are changed or added to form a Layer II encoder are shown highlighted in Figure 9.16. The same analysis filter bank is used for both Layers I and II but in Layer II, three blocks of 12×32 subband samples are combined to form a *superblock*. This means a Layer II audio frame is effectively triple the size of a Layer I frame and contains the coded data for 1152 PCM samples compared with 384 samples for Layer I. The number of samples used in the FFT calculation is also increased to 1024 so that the power spectral density used in the psychoacoustic model is representative of the audio samples to be coded. A scalefactor is still calculated for each set of 12 samples in a subband so there are

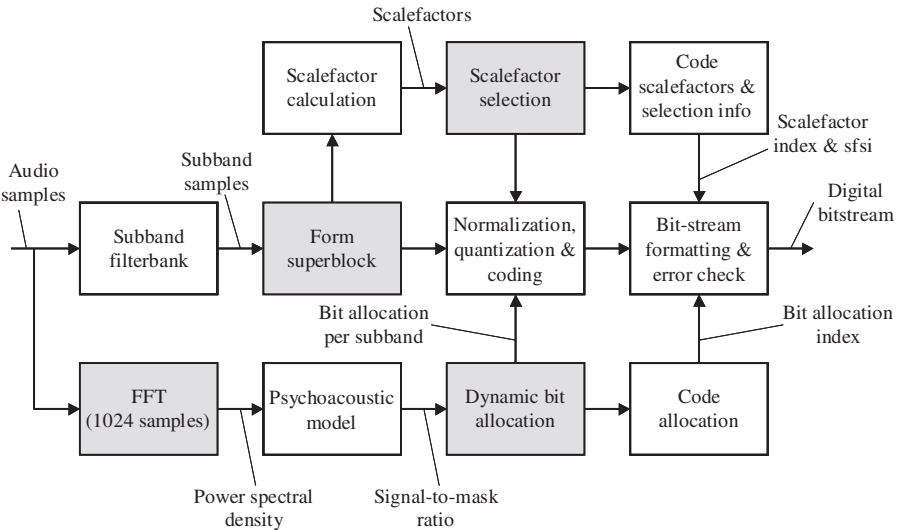


Figure 9.16 Basic block diagram of a Layer II encoder.

now three scalefactors for each subband in the superblock. Since there is often some redundancy between these three scalefactors, a more sophisticated method of scalefactor coding is adopted in Layer II. The final major modification occurs in the bit allocation procedure where the number of allowable quantizer step sizes is reduced for mid and high frequency subbands resulting in a saving in the bits used for the bit allocation code words.

9.2.1. Analysis Filterbank

The same analysis filterbank is used in both Layers I and II (see Section 9.1.1).

9.2.2. Scalefactor Calculation

As in Layer I, a *scalefactor* is calculated for each set of 12 subband samples by finding the maximum of the absolute values of the 12 subband samples and then taking the lowest value from Table 9.2 that is greater than this maximum value. The scalefactor index is then defined as the index from Table 9.2 that corresponds to this value.

9.2.3. Coding of Scalefactors

For a Layer II frame the audio data contains information from a *superblock* that consists of three consecutive blocks of 12 subband samples for each subband. This means that three scalefactors are calculated for each subband in the superblock. Often there is some redundancy between these three scalefactors and for this reason a Layer II encoder employs the following method to efficiently code the scalefactor information for a superblock.

The first step in coding the scalefactors is to calculate the two scalefactor differences, $dscf_1$ and $dscf_2$, using the following two equations:

$$\begin{aligned} dscf_1 &= scf_1 - scf_2 \\ dscf_2 &= scf_2 - scf_3 \end{aligned} \tag{9.23}$$

where scf_1 , scf_2 , and scf_3 are the successive scalefactor indices for the three blocks calculated as described in Section 9.2.2. The scalefactor differences are then categorized into one of five classes using the criteria described in Table 9.6.

The pair of scalefactor difference classes for each subband is then used to determine the method for transmitting the scalefactors from Table 9.7.

In Table 9.7, the third column titled “scalefactor used in encoder” indicates the scalefactors that are actually used in the later encoding procedures. The numbers in this column, indicate which scalefactor is used with 1, 2, and 3 indicating the scalefactors from the first, second, and third blocks in the superblock respectively and 4 indicating the maximum of the three scalefactor values (i.e. the scalefactor with the minimum scalefactor index).

Table 9.6 Scalefactor difference classes.

Class	$dscf$
1	$dscf \leq -3$
2	$-3 < dscf < 0$
3	$dscf = 0$
4	$0 < dscf < 3$
5	$dscf \geq 3$

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.7 Scalefactor transmission patterns and selection information.

Class ₁	Class ₂	Scalefactors used in encoder	Transmission pattern	Selection information
1	1	1 2 3	1 2 3	0
1	2	1 2 2	1 2	3
1	3	1 2 2	1 2	3
1	4	1 3 3	1 3	3
1	5	1 2 3	1 2 3	0
2	1	1 1 3	1 3	1
2	2	1 1 1	1	2
2	3	1 1 1	1	2
2	4	4 4 4	4	2
2	5	1 1 3	1 3	1
3	1	1 1 1	1	2
3	2	1 1 1	1	2
3	3	1 1 1	1	2
3	4	3 3 3	3	2
3	5	1 1 3	1 3	1
4	1	2 2 2	2	2
4	2	2 2 2	2	2
4	3	2 2 2	2	2
4	4	3 3 3	3	2
4	5	1 2 3	1 2 3	0
5	1	1 2 3	1 2 3	0
5	2	1 2 2	1 2	3
5	3	1 2 2	1 2	3
5	4	1 3 3	1 3	3
5	5	1 2 3	1 2 3	0

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.8 Meaning of the scalefactor selection information.

Selection information	Meaning
00	The first, second, and third scalefactor transmitted should be allocated to blocks 1, 2, and 3, respectively.
01	The first scalefactor transmitted should be allocated to blocks 1 and 2 and the second to block 3.
10	The scalefactor transmitted should be allocated to all three blocks.
11	The first scalefactor transmitted should be allocated to block 1 and the second to blocks 2 and 3.

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

The third column of Table 9.7 shows that there is often some duplication of the scalefactors across the three blocks and so it is not always necessary to transmit all three scalefactors for each subband in a frame. For this reason only the scalefactors indicated by the transmission pattern in the next column of Table 9.7 are actually transmitted. The next column of Table 9.7 contains the *scalefactor selection information* which is a number between 0 and 3 that is used to indicate how the transmitted scalefactor indices are to be allocated to the three blocks during the decoding process. The meaning of the four values of the scalefactor selection information is shown in Table 9.8.

The scalefactor selection information is transmitted as a two bit unsigned integer for those subbands that are allocated with more than zero bits in the following bit allocation procedure.

9.2.4. Dynamic Bit Allocation

The total number of bits available for each audio frame, cb , is determined from the number of 8-bit slots that are to be transmitted for that frame (see Section 9.2.8). Not all of the bits in each frame are available to transmit subband sample and scalefactor information. The available number of bits can be determined by subtracting from the total number of bits available, cb , the number of bits required to transmit the header, $bhdr$, (32 bits), the CRC check word if used, $bcrc$, (16 bits), the bit allocation for each subband, bba , and any ancillary data, $banc$. So the number of bits available for subband sample and scalefactor information, adb , is given by the following equation:

$$adb = cb - (bhdr + bcrc + bba + banc) \quad (9.24)$$

The available bits are then allocated to the subbands using the principle of minimizing the total noise-to-mask ratio of the frame. The number of bits allocated to one sample can range from 0 to 16 bits with the exception of 1 bit per sample, which

is not allowed. For a Layer II encoder certain restrictions are made on the number of bits that can be allocated to each subband. These restrictions depend on the input sampling rate, the output bit rate and the subband. Tables and show the allowable number of bits for each subband for the various input sampling rates and output bit rates.

The procedure to find the number of bits allocated to each subband begins by finding the *mask-to-noise-ratio* (MNR) for each subband. The MNR is calculated by subtracting the signal-to-mask ratio from the signal-to-noise ratio for each subband and is given by

$$\text{MNR} = \text{SNR} - \text{SMR} \quad (9.25)$$

The signal-to-mask ratio is the output of the psychoacoustic model and the signal-to-noise ratio depends on the number of bits allocated to the subband. The relationship between SNR and bits allocated to a subband is given in Table 9.9. Note that the number of bits allocated in column 1 of Table 9.9 refers to the number of bits for the group of three subband samples in each granule (see Section 9.2.6).

Table 9.9 SNR per number of bits allocated to each subband for a Layer II encoder.

No. of bits	SNR (dB)	<i>A</i>	<i>B</i>
0	0.00		
5	7.00	0.750000000	−0.250000000
7	11.00	0.625000000	−0.375000000
9	16.00	0.875000000	−0.125000000
10	20.84	0.562500000	−0.437500000
12	25.28	0.937500000	−0.062500000
15	31.59	0.968750000	−0.031250000
18	37.75	0.984375000	−0.015625000
21	43.84	0.992187500	−0.007812500
24	49.89	0.996093750	−0.003906250
27	55.93	0.998046875	−0.001953125
30	61.96	0.999023438	−0.000976563
33	67.98	0.999511719	−0.000488281
36	74.01	0.999755859	−0.000244141
39	80.03	0.999877930	−0.000122070
42	86.05	0.999938965	−0.000061035
45	92.01	0.999969482	−0.000030518
48	98.01	0.999984741	−0.000015259

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

The allocation of bits to each subband is an iterative process that begins by allocating zero bits for the scalefactors and subband samples of each subband. The iterative process then continues using the following steps:

1. Use the number of bits allocated to each subband to determine the SNR for each subband and then calculate the MNR for each subband.
2. Determine the subband that has the minimum MNR.
3. The number of bits allocated to this subband is then increased to the next highest allowable number of bits.
4. The number of bits required to transmit the samples, $bspl$, is updated to the number of bits required in step 2. In addition, if $bspl$ has increased from zero to five then the number of bits required to transmit the scalefactors, $bscf$, and the scalefactor selection information, $bsel$, is updated for this subband.
5. Recalculate the number of bits available for subband sample and scalefactor information, adb , using the following equation:
$$adb = cb - (bhdr + bcrc + bbal + bsel + bscf + bspl + banc)$$
6. If adb is greater than any possible increase of $bsel + bscf + bspl$ the iterative procedure continues from step 1.

9.2.5. Coding of Bit Allocation

The bit allocation information for each subband is coded as an unsigned integer, MSB first. The value of this integer is taken as the allocation index from Tables A.5, A.6, A.7 or A.8. The number of bits used to code the allocation index is indicated by the column labeled $nbal$ in these tables. The number of bits per group of three subband samples that corresponds to each value of the allocation index is shown in the remaining columns.

9.2.6. Quantization and Coding of Subband Samples

The subband samples are coded using a linear quantizer with a symmetric zero representation. This type of quantizer guarantees that small values close to zeros are quantized to zero. The quantization process is defined by the following steps:

1. Normalize the subband sample by dividing its value by the scalefactor to obtain the normalize sample X .
2. Calculate the quantized value Q using the following formula:

$$Q = AX + B$$

where the values for the quantization coefficients A and B depend on the number of bits allocated to the subband and are given in Table 9.9.

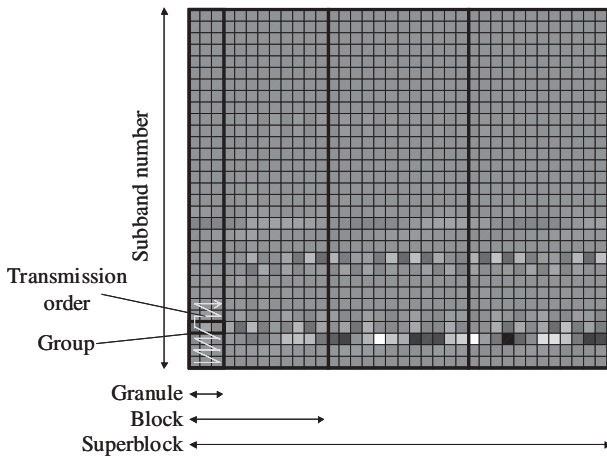


Figure 9.17 Subband sample transmission order for a single channel Layer II frame.

3. Find Q_N by taking the N most significant bits of Q , where N is the number of bits allocated in the previous bit allocation process and Q is stored as a two's complement binary fraction.
4. Invert the most significant bit of Q_N in order to avoid the code word containing all 1s.

For a Layer II coder, each block of subband samples is further subdivided into four *granules* as shown in Figure 9.17. Each shaded square in Figure 9.17 represents the value of a subband sample with gray indicating zero, white indicating a large positive value, and black indicating a large negative value. Each column of shaded squares represents one set of 32 subband sample outputs from the analysis filter and one granule consists of three of these sets of 32 subband samples. The Layer II coding algorithm specifies that subband samples be transmitted as sets of three samples for each subband in each granule. If two channels are to be coded, the two sets of three samples for each channel in each subband are transmitted consecutively.

If the number of bits allocated to the subband is 5, 7, or 10 then the code words corresponding to the set of three samples are treated as unsigned integers and *grouped* together into a single larger code word. Otherwise, each sample is transmitted as a separate code word.

If grouping is required, the three smaller code words denoted by w_a , w_b , and w_c are treated as unsigned integers and combined into a single larger code word using the following equations:

$$\begin{aligned}
 w_5 &= 9w_c + 3w_b + w_a \\
 w_7 &= 25w_c + 5w_b + w_a \\
 w_{10} &= 81w_c + 9w_b + w_a
 \end{aligned}
 \tag{9.26}$$

Then w_5 , w_7 , and w_{10} are transmitted as 5, 7, and 10-bit unsigned integers, respectively, MSB first.

EXAMPLE 9.4

To quantize three consecutive subband samples using five bits, the first step is to normalize the subband values by the scalefactor to give a value for X . Since the scalefactor must always be larger than the maximum sample value in each subband, the possible range of values for X is defined by $-1 < X < 1$. The value of X is then used to calculate a value for Q using the equation

$$Q = 0.75 X - 0.25$$

and the possible range of values for Q is then defined by $-1 < Q < 0.5$.

If Q is then stored as a two's complement binary fraction and Q_2 is taken as the first two bits of this binary fraction, there are only three different possible values for Q_2 . Table 9.10 shows the range of values for X and Q that correspond to each of these three values of Q_2 . The last column of Table 9.10 shows the code word that is grouped together with two other code words to form a single larger code word after the most significant bit of Q_2 has been inverted.

Table 9.10 Ranges and code words for a 2-bit quantizer.

Range for X	Range for Q	Q_2	Code word
$-1 < X < \frac{-1}{3}$	$-1 < Q < -0.5$	10	00
$\frac{-1}{3} \leq X < \frac{1}{3}$	$-0.5 \leq Q < 0.0$	11	01
$\frac{1}{3} \leq X < 1$	$0.0 \leq Q < 0.5$	10	00

Then three consecutive code words produced in this way, w_a , w_b , and w_c , are treated as unsigned integers (i.e. with values of 0, 1, or 2) and grouped together to form a single code word using the equation

$$w_5 = 9w_c + 3w_b + w_a$$

This equation results in a value of w_5 between 0 and 26 that is transmitted as a 5-bit unsigned integer, MSB first. ■

9.2.7. Ancillary Data

See Section 9.1.6.

9.2.8. Formatting

The output of the audio encoder is transmitted in frames. In a Layer II bit stream, each frame contains the information required to decode 1152 samples of the original audio signal. Each frame must consist of an integer number of *slots*. In Layer II, a

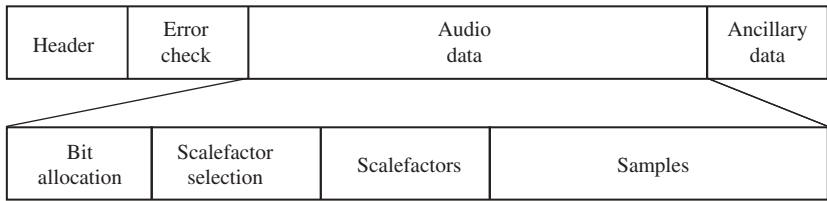


Figure 9.18 Format of a Layer II audio frame.

slot is defined as 8 bits and the number of slots in a frame, N_s , can be calculated using the following formula:

$$N_s = \text{int} \left[\frac{1152R/F_s}{8} \right] \quad (9.27)$$

where R is the output bit rate and F_s is the input sampling frequency. As for a Layer I frame, the number of slots in each frame may vary between N_s and $N_s + 1$. Figure 9.18 shows the general format of a Layer II frame. The bit stream syntax required for each audio frame is described later in Section 9.4.

9.3. JOINT STEREO CODING

The ability of the human auditory system to determine the location of sound sources depends on interaural level differences and interaural time differences. For frequencies above 2 kHz, it is the level differences between the envelope of the left and right signals that determines the perceived lateral position of the sound source. For Layer I and II encoders an optional joint stereo coding mode known as *intensity stereo coding* is available to exploit this limitation of the human auditory system.

In intensity stereo mode, the samples for the left and right channels for some subbands are added and transmitted as a single channel. The bit allocation, quantization and coding of the combined subband samples is performed as for a single channel but the scalefactor information is transmitted for both the left and right channels. In this way, the decoded left and right signals are identical except for the level. However, the ability of the human auditory system to perceive the location of the sound source is unaffected by this approximation.

The intensity stereo coding mode can be applied in one of four different configurations. The difference between the configurations is simply the lowest frequency subband that is coded using the intensity stereo method. The four possible choices for the lowest subband coded in the intensity stereo mode are subbands 4, 8, 12, or 16.

The first step in the intensity stereo process is to determine the number of subbands to be code in intensity stereo mode. For these subbands the left and right samples are added and the combined samples are scaled in the same way as for subbands from a single channel. It is important to note however that the scalefactor used in this step is not transmitted. Instead, the originally determined scalefactors for the left and right channels are transmitted. The combined and scaled samples are then quantized and coded as for a single channel using the higher of the bit allocations from the left and right channels.

Use of the intensity stereo mode generally results in a bit-rate saving of between 10 and 30 kbit/s for no perceivable degradation of audio quality and requires only a small increase in encoder and decoder complexity.

9.4. MPEG-1 SYNTAX

The syntax of the MPEG-1 audio bit stream is specified in this section. The order of arrival of the bits is defined using the C-like methodology described in Chapter 5 for the simple video coder syntax.

9.4.1. Audio Sequence Layer

The syntax specification for an MPEG-1 audio bit stream is given in Table 9.11. The bitstream consists of consecutive audio frames.

9.4.2. Audio Frame

The syntax specification for an MPEG-1 audio frame is given in Table 9.12.

Table 9.11 Syntax specification for an MPEG-1 audio bitstream.

	Syntax	Bits	Mnemonic
1	audio_sequence()		
2	{		
3	while (nextbits()==syncword){		
4	frame()		
5	}		
6	}		

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.12 Syntax specification for an MPEG-1 audio frame.

	Syntax	Bits	Mnemonic
1	frame()		
2	{		
3	header()		
4	error_check()		
5	audio_data()		
6	ancillary_data()		
7	}		

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

9.4.3. Header

The header field contains information necessary for the decoder to maintain synchronization of the incoming bit stream and parameters that specify aspects of the audio service being transmitted. The syntax specification of the header field is given in Table 9.13

A description of the bit stream elements contained in a header field is given in Table 9.14.

The meaning of the **layer** code word is specified in Table 9.15.

The meaning of the **bitrate_index** code word is specified in Table 9.16. The specified bit rate is the total bit rate for all channels regardless of the coding mode. The “free” format is used to allow a fixed bit rate that is not specified in Table 9.16 to be used. In “free” format, the maximum bit rate allowed is 448 and 384 in Layers I and II, respectively. A continuously variable bit rate is not allowed for Layers I and II.

For Layer II, not all combinations of total bit rate and mode are allowed. The combinations that are allowed are shown in Table 9.17.

The meaning of the **sampling_frequency** code word is specified in Table 9.18.

The meaning of the **mode** code word is specified in Table 9.19.

Table 9.13 Syntax specification of the header field.

	Syntax	Bits	Mnemonic
1	header()		
2	{		
3	syncword	12	bslbf
4	ID	1	bslbf
5	layer	2	bslbf
6	protection_bit	1	bslbf
7	bitrate_index	4	bslbf
8	sampling_frequency	2	bslbf
9	padding_bit	1	bslbf
10	private_bit	1	bslbf
11	mode	2	bslbf
12	mode_extension	2	bslbf
13	copyright	1	bslbf
14	original/copy	1	bslbf
15	emphasis	2	bslbf
16	}		

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.14 Description of the bit stream elements in a header field.

Element	Description
syncword	Used to detect the start of the header field. The value of the syncword is always 0×FFF.
ID	A value of “1” indicates the following bit stream was coded using MPEG-1. The value “0” is reserved.
layer	Indicates the layer used to produce the following bit stream (see Table 9.15).
protection_bit	A value of “1” indicates that the error_check field follows the header.
bitrate_index	Specifies the output bit rate of the coder (see Table 9.16).
sampling_frequency	Specifies the input sampling frequency (see Table 9.18).
padding_bit	A value of “1” indicates that the following audio frame contains an extra slot (see Section 9.1.7).
private_bit	Flag for private use.
mode	Specifies the coding mode (see Table 9.19).
mode_extension	Specifies the highest frequency subband that is not included in intensity stereo coding (see Table 9.20).
copyright	A value of “1” indicates that the following bit stream is protected by copyright.
original/copy	A value of “1” indicates that the following bit stream is an original bit stream.
emphasis	Specifies the type of emphasis/deemphasis used (see Table 9.21).

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

In intensity stereo mode the code word **mode_extension** specifies the highest frequency subband that is not included in intensity stereo coding. The meaning of the **mode_extension** code word is specified in Table 9.20.

The meaning of the **emphasis** code word is specified in Table 9.21.

Table 9.15 Meaning of Layer.

layer	Meaning
“11”	Layer I
“10”	Layer II
“01”	Layer III
“00”	Reserved

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.16 Meaning of `bitrate_index`.

bitrate_index	Specified bitrate (kbits/s)	
	Layer I	Layer II
“0000”	free	free
“0001”	32	32
“0010”	64	48
“0011”	96	56
“0100”	128	64
“0101”	160	80
“0110”	192	96
“0111”	224	112
“1000”	256	128
“1001”	288	160
“1010”	320	192
“1011”	352	224
“1100”	384	256
“1101”	416	320
“1110”	448	384
“1111”	Forbidden	Forbidden

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.17 Allowed modes for each bit rate of a Layer II encoder.

Bit rate (kbits/s)	Allowed modes
Free format	All modes
32	Single_channel
48	Single_channel
56	Single_channel
64	All modes
80	Single_channel
96	All modes
112	All modes
128	All modes
160	All modes
192	All modes
224	Stereo, intensity stereo, dual channel
256	Stereo, intensity stereo, dual channel
320	Stereo, intensity stereo, dual channel
384	Stereo, intensity stereo, dual channel

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.18 Meaning of `sampling_frequency`.

<code>sampling_frequency</code>	Specified frequency (kHz)
"11"	44.1
"10"	48
"01"	32
"11"	Reserved

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.19 Meaning of mode.

<code>mode</code>	Specified mode
"11"	Stereo
"10"	Intensity_stereo
"01"	Dual_channel
"11"	Single_channel

© This Table is based on AS/NZS 4230.3:1994.
Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.20 Meaning of `mode_extension`.

<code>mode_extension</code>	Meaning
"00"	Subbands 4 – 31 in intensity_stereo, bound = 4
"10"	Subbands 8 – 31 in intensity_stereo, bound = 8
"01"	Subbands 12 – 31 in intensity_stereo, bound = 12
"11"	Subbands 16 – 31 in intensity_stereo, bound = 16

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.21 Meaning of emphasis.

<code>emphasis</code>	Specified emphasis
"00"	None
"10"	50/15 μ s
"01"	Reserved
"11"	CCITT J.17

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.22 Syntax specification of the error_check field.

	Syntax	Bits	Mnemonic
1	error_check()		
2	{		
3	if (protection_bit==0)		
4	crc_check	16	rpchof
5	}		

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

9.4.4. Error Check

The syntax specification of the error_check field is given in Table 9.22.

The **crc_check** code word contains a 16 bit parity-check word. This code word is transmitted using the remainder polynomial coefficients, highest order first (rpchof) data type. Further details of the error checking procedure can be found in Ref. [1].

9.4.5. Audio Data, Layer I

The syntax specification of the audio_data field in Layer I is given in Table 9.23.

A description of the bit stream elements contained in an audio_data field for Layer I is given in Table 9.24.

9.4.6. Audio Data, Layer II

The syntax specification of the audio_data field in Layer II is given in Table 9.25.

A description of the bit stream elements contained in an audio_data field for Layer II is given in Table 9.26.

9.5. MPEG-1 LAYER I,II DECODERS

9.5.1. Bit Allocation Decoding

For a Layer I decoder, the 4-bit allocation parameter is read for all subbands. If the coding mode is joint stereo then only one bit allocation parameter is read for each subband above the subband indicated by the **mode_extension** parameter.

For a Layer II decoder, the number of bits used for the bit allocation parameter varies for different input sampling rates and output bit rates and can be found using tables in Appendix. The output bit rate used to determine the appropriate table is the

Table 9.23 Syntax specification of the audio_data field in Layer I.

	Syntax	Bits	Mnemonic
1	audio_data(){		
2	for (sb=0; sb<bound; sb++){		
3	for (ch=0; ch<nch; ch++){		
4	allocation[ch][sb]	4	uimbsf
5	for (sb=bound; sb<32; sb++){		
6	allocation[0][sb]	4	uimbsf
7	allocation[1][sb]=allocation[0][sb]		
8	}		
9	for (sb=0; sb<32; sb++){		
10	for (ch=0; ch<nch; ch++){		
11	if (allocation[ch][sb]!=0)		
12	scalefactor[ch][sb]	6	uimbsf
13	for (s=0; s<12; s++){		
14	for (sb=0; sb<bound; sb++){		
15	for (ch=0; ch<nch; ch++){		
16	if (allocation[ch][sb]!=0)		
17	sample[ch][sb][s]	2..15	uimbsf
18	for (sb=bound; sb<32; sb++){		
19	if (allocation[ch][sb]!=0)		
20	sample[0][sb][s]	2..15	uimbsf
21	}		
22	}		

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

bit rate for one channel so if the coding mode is not single channel then the output bit rate indicated in the frame header should be divided by two to find the bit rate per channel. The number of bits to read for each subband is indicated by the value of *nbal* in the appropriate table in Appendix. This allocation parameter is treated as

Table 9.24 Description of the bitstream elements in an audio_data field for Layer I.

Element	Description
allocation[ch][sb]	Specifies the bits per sample used to code the subband samples in subband sb of channel ch (see Section 9.1.4).
scalefactor[ch][sb]	Specifies the scalefactor used in subband sb of channel ch (see Section 9.1.2).
sample[ch][sb][s]	The coded value for sample s in subband sb of channel ch (see Section 9.1.6)

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

Table 9.25 Syntax specification of the audio_data field in Layer II.

	Syntax	Bits	Mnemonic
1	audio_data(){		
2	for (sb=0; sb<bound; sb++){		
3	for (ch=0; ch<nch; ch++){		
4	allocation[ch][sb]	2..4	uimbsbf
5	for (sb=bound; sb<sblimit; sb++){		
6	allocation[0][sb]	2..4	uimbsbf
7	allocation[1][sb]=allocation[0][sb]		
8	}		
9	for (sb=0; sb<sblimit; sb++){		
10	for (ch=0; ch<nch; ch++){		
11	if (allocation[ch][sb]!=0)		
12	scfsi[ch][sb]	2	uimbsbf
13	for (sb=0; sb<32; sb++){		
14	for (ch=0; ch<nch; ch++){		
15	if (allocation[ch][sb]!=0)		
16	if (scfsi[ch][sb]==0){		
17	scalefactor[ch][sb][0]	6	uimbsbf
18	scalefactor[ch][sb][1]	6	uimbsbf
19	scalefactor[ch][sb][2]	6	uimbsbf
20	}		
21	if ((scfsi[ch][sb]==1) (scfsi[ch][sb]==3)){		
22	scalefactor[ch][sb][0]	6	uimbsbf
23	scalefactor[ch][sb][1]	6	uimbsbf
24	}		
25	if ((scfsi[ch][sb]==2)		
26	scalefactor[ch][sb][0]	6	uimbsbf
27	}		
28	for (gr=0; gr<12; gr++){		
29	for (sb=0; sb<bound; sb++){		
30	for (ch=0; ch<nch; ch++){		
31	if (allocation[ch][sb]!=0){		
32	if (grouping[ch][sb])		
33	samplecode[ch][sb][gr]	5..10	uimbsbf
34	else		
35	for (s=0; s< 3; s++){		
36	sample[ch][sb][3*gr+s]	3..16	uimbsbf
37	}		
38	for (sb=bound; sb<sblimit; sb++){		
39	if (allocation[0][sb]!=0){		
40	if (grouping[0][sb])		
41	samplecode[0][sb][gr]	5..10	uimbsbf
42	else		

Table 9.25 (Continued)

	Syntax	Bits	Mnemonic
43	for (s=0; s< 3; s++)		
44	sample[0][sb][3*gr+s]	3..16	uimsbf
45	}		
46	}		
47	}		

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

an unsigned integer and used as an index into the remaining columns of the table. The value indicated by this index represents the number of bits transmitted in the corresponding subband. This number of bits can then in turn be used as an index into Table 9.27 that shows the number of bits used to code the quantized samples and the requantization coefficients C and D .

9.5.2. Scalefactor Selection Information Decoding

For a layer II decoder the 2-bit scalefactor selection information (*scfsi*) code word is read for all subbands that have a nonzero bit allocation. The meaning of the *scfsi* code word can be found in Table 9.8.

9.5.3. Scalefactor Decoding

For a Layer I decoder, a 6-bit scalefactor code word for each subband with a nonzero bit allocation is read from the bit stream and used as an index into Table 9.2 to find the scalefactor for the corresponding subband.

Table 9.26 Description of the bit stream elements in an audio_data field for Layer II.

Element	Description
allocation[ch][sb]	Specifies the bits per sample used to code the subband samples in subband sb of channel ch (see Section 9.2.4).
scfsi[ch][sb]	Specifies the number of scalefactors transferred for subband sb of channel ch (see Section 9.2.3).
scalefactor[ch][sb][p]	Specifies the scalefactor used subband sb of channel ch for part p of the frame (see Section 9.2.3)
samplecode[ch][sb][gr]	The code word for the three consecutive sample values in granule gr in subband sb of channel ch if grouping is used (see Section 9.2.6).
sample[ch][sb][s]	The coded value for sample s in subband sb of channel ch (see Section 9.2.6).

Table 9.27 Classes of quantization for a Layer II decoder.

No. of bits	C	D
5	1.33333333333	0.50000000000
7	1.60000000000	0.50000000000
9	1.14285714286	0.25000000000
10	1.77777777777	0.50000000000
12	1.06666666666	0.12500000000
15	1.03225806452	0.06250000000
18	1.01587301587	0.03125000000
21	1.00787401575	0.01562500000
24	1.00392156863	0.00781250000
27	1.00195694716	0.00390625000
30	1.00097751711	0.00195312500
33	1.00048851979	0.00097656250
36	1.00024420024	0.00048828125
39	1.00012208522	0.00024414063
42	1.00006103888	0.00012207031
45	1.00003051851	0.00006103516
48	1.00001525902	0.00003051758

© This Table is based on AS/NZS 4230.3:1994. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

For a Layer II decoder, the number of 6-bit scalefactor code words corresponding to each subband with a nonzero bit allocation is indicated by the scalefactor selection information. These scalefactor code words are read from the bit stream and used as an index into Table 9.2 to determine the three scalefactors for the corresponding subband.

9.5.4. Requantization of Subband Samples

9.5.4.1. Layer I

The number of bits indicated by the bit allocation parameter is read from the bit stream. This N -bit code word is then used to find the value of the requantized and rescaled subband sample using the following steps:

1. Invert the most significant bit of the N -bit code word to obtain the 2's complement fractional number R_N .
2. Calculate the requantized value R using the following formula:

$$R = \frac{2^N}{2^N - 1} (R_N + 2^{1-N})$$

3. Rescale the subband sample by multiplying R by the scalefactor for the corresponding subband to obtain the rescaled subband sample S .

9.5.4.2. Layer II

If the number of bits allocated to the subband is 5, 7, or 10 then the code words corresponding to the set of three consecutive samples are grouped together into a single larger code word. The algorithm used to convert this group code word back into three individual code words is defined by the following MATLAB code. The variable *nlevels* has the value 3, 5, or 9 if the number of bits allocated to the subband is 5, 7, or 10, respectively.

```
for i = 1:3
    code_word(i) = mod(group_code,nlevels)
    group_code = floor(group_code/nlevels);
end
```

Then, for each individual *N*-bit code word, the following steps are used to find the value of the requantized and rescaled subband sample:

1. Invert the most significant bit of the *N*-bit code word to obtain the 2's complement fractional number R_N .
2. Calculate the requantized value R using the following formula:

$$R = C(R_N + D)$$

where the values for the requantization coefficients C and D depend on the number of bits allocated to the subband and are given in Table 9.27.

3. Rescale the subband sample by multiplying R by the scalefactor for the corresponding subband to obtain the rescaled subband sample S .

9.5.5. Synthesis Filterbank

The synthesis filterbank used in the Layer I,II encoders is based on the efficient implementation of a cosine modulated filterbank proposed by Joseph Rothweiler in 1983. A detailed explanation of this filter bank technique is given in Section 8.5 of Chapter 8.

9.6. MPEG-2

The DVB guidelines state that an IRD need only be able to decode a stereo pair from an MPEG-2 BC bit stream. So this section describes the frame formatting and matrixing procedures required to produce an MPEG-1 compatible frame within an MPEG-2 bit stream.

9.6.1. Backwards-Compatible MPEG-2 Frame Formatting

If a 5.1 channel signal is to be transmitted using an MPEG-2 encoder and backwards compatibility with an MPEG-1 decoder is required, the first step is to down-mix

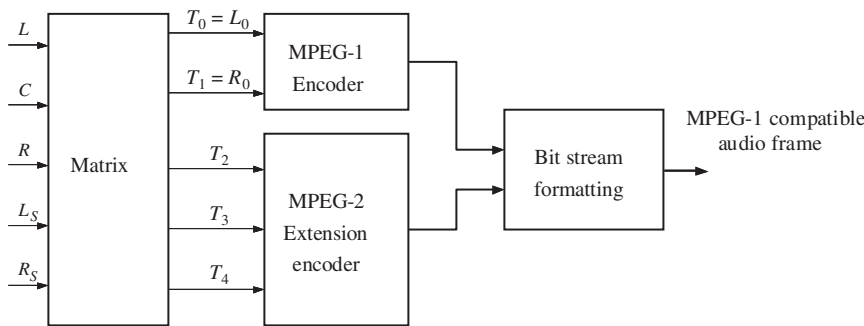


Figure 9.19 The down-mixed stereo pair is encoded using an MPEG-1 encoder and the remaining multichannel data is encoded using an MPEG-2 extension encoder.

the five input channels into a basic stereo pair. This down-mixing process is usually referred to as *matrixing* and the resulting stereo pair is denoted by L_0 and R_0 . This stereo pair is then allocated to *transmission channels* T_0 and T_1 and encoded using an MPEG-1 encoder. Transmission channels T_2 , T_3 , and T_4 contain the audio data required to reconstruct the remaining multichannel signal and are coded using an MPEG-2 *extension encoder* as shown in Figure 9.19.

Backwards compatibility with an MPEG-1 decoder is achieved by placing the data for transmission channels T_0 and T_1 in the audio data field of an MPEG-1 frame and all the multichannel extension data in the ancillary data field of the frame as shown in Figure 9.20. An MPEG-1 decoder simply ignores the multichannel data in the ancillary data field and decodes the stereo pair L_0/R_0 . An MPEG-2 decoder decodes both the stereo pair and the multichannel data and performs a dematrixing procedure to obtain the decoded 5.1 multichannel signals.

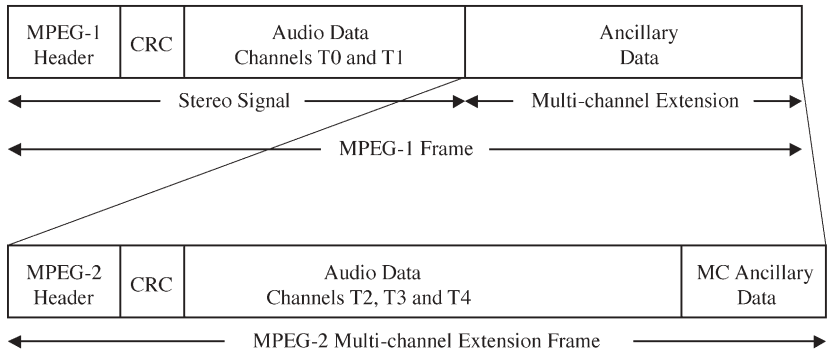


Figure 9.20 Data format of an MPEG-2 audio frame that is backwards compatible with an MPEG-1 decoder.

Table 9.28 Filter coefficients for the filter used in matrixing procedure 2.

Sampling frequency (kHz)	a_0	b_0	b_1	b_2
32	486	2048	−471	370
44.1	295	2048	−1394	521
48	294	2048	−1388	520

© This Table is based on AS/NZS 13818.3:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

9.6.2. Matrixing Procedures for Backwards Compatibility

The MPEG-2 standard defines four possible matrixing procedures that can be used to obtain the L_0/R_0 stereo pair from a 5.1 channel signal. The equations used for procedures 0, 1, and 3 are

$$L_0 = \alpha (L + \beta C + \gamma L_S) \quad (9.28)$$

$$R_0 = \alpha (R + \beta C + \gamma R_S) \quad (9.29)$$

Matrixing procedure 2 can be used to produce a L_0/R_0 stereo pair that is compatible with a Dolby Pro Logic decoder and the equations used for this procedure are

$$L_0 = \alpha (L + \beta C + \gamma jS) \quad (9.30)$$

$$R_0 = \alpha (R + \beta C + \gamma jS) \quad (9.31)$$

To obtain the signal jS used in Equations (9.30) and (9.31), half Dolby B-type encoding is applied to the signals L_S and R_S followed by a 90° phase shift. The monophonic component of the resulting signals is then calculated and limited in bandwidth to the range 100–7 kHz to produce the surround signal jS . The transfer function of the bandwidth-limiting filter is given in Equation (9.32) and the coefficients to be used in this equation are given in Table 9.28 for each input sampling frequency.

$$H(z) = \frac{a_0(1 + 2z^{-1} + z^{-2})}{b_0 + b_1z^{-1} + b_2z^{-2}} \quad (9.32)$$

The attenuation factors, α , β , and γ , differ for each procedure and are given in Table 9.29.

9.7. SUMMARY

The DVB digital television standard prescribes the use of an MPEG-1 or backwards compatible MPEG-2 bit stream to transmit audio information. The MPEG-1 audio standard defines a multilayer coding algorithm with each additional layer providing increased complexity and coding efficiency. The MPEG-2 BC audio standard defines a mechanism for transmitting a multichannel audio signal that can be decoded by an MPEG-1 decoder. The encoding algorithm, syntax, and decoding algorithm for the first two layers of the MPEG-1 standard and the matrixing operations required to allow an MPEG-1 decoder to decode an MPEG-2 bit stream are described in this chapter.

Table 9.29 Attenuation factors for each matrixing procedure.

Matrixing procedure	α	β	γ
0	$\frac{1}{1+\sqrt{2}}$	$\frac{1}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}$
1	$\frac{1}{1.5+0.5\sqrt{2}}$	$\frac{1}{\sqrt{2}}$	$\frac{1}{2}$
2	$\frac{1}{1+\sqrt{2}}$	$\frac{1}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}$
3	1	0	0

© This Table is based on AS/NZS 13818.3:2002. Permission to reprint has been granted by SAI Global Ltd. The standard can be purchased online at <http://www.sai-global.com>.

PROBLEMS

- 9.1** Draw a simple block diagram of an MPEG-1 encoder that includes the following system components and briefly explain the function of each of these components:
- (a) Analysis filterbank
 - (b) Psychoacoustic model
 - (c) Dynamic bit allocation
 - (d) Normalization quantization and coding
 - (e) Bitstream formatting
- 9.2** Briefly explain the four audio coding modes that are possible with an MPEG-1 audio coder.
- 9.3** Explain how a scalefactor is calculated for each of the subbands in an audio block.
- 9.4** Explain why a psychoacoustic model is used to calculate a signal-to-mask ratio value for each subband.
- 9.5** Explain what is meant by the following terms when used in the context of Psychoacoustic model 1 in the MPEG-1 encoder:
- (a) Maximum sound pressure level in each subband
 - (b) Threshold-in-quiet
 - (c) Tonal and Nontonal components
 - (d) Individual masking threshold
 - (e) Global masking threshold
 - (f) Signal-to-mask ratio
- 9.6** Explain how the global masking threshold curve is calculated from the individual masking curves for each masker.
- 9.7** Explain how the minimum masking threshold for each subband is calculated from the global masking curve.
- 9.8** Describe the process for allocating the number of bits to be used to quantize the subband samples in each subband for a Layer I coder.
- 9.9** Describe the process for quantizing and coding subband samples in each subband for a Layer I coder.
- 9.10** Complete the following table by determining the actual bits to be transmitted to the decoder for the given normalized subband sample and *bspl* values.

Subband sample	<i>bspl</i>	Code word
0.2311	3	
−0.6068	5	
0.4860	7	
−0.8913	9	

- 9.11 Calculate the number of slots per frame for a Layer I coder with an input sampling rate of 48 kHz and output bitrate of 128 kbit/s.
- 9.12 With the aid of a diagram describe the format of an MPEG-1 Layer I audio frame.
- 9.13 Briefly explain the major differences between the MPEG-1 Layer I encoding algorithm and the MPEG-1 layer II encoding algorithm.
- 9.14 Complete the following table by determining the pair of scalefactor difference classes, the scalefactors to be used in the encoder, the transmission pattern and the scalefactor selection information for each set of three scalefactor indices.

Scalefactor indices	Class ₁	Class ₂	Scalefactors used in encoder	Transmission pattern	Selection information
5,10,20					
32,34,36					
20,25,26					
16,18,17					
48,32,34					
36,30,24					

- 9.15 Describe the process for allocating the number of bits to be used to quantize the subband samples in each subband for a Layer II coder.
- 9.16 Describe the process for quantizing and coding subband samples in each subband for a Layer II coder.
- 9.17 With the aid of a diagram, explain what is meant by the following terms when used in the context of quantizing and coding subband samples in an MPEG-1 layer II encoder:
- (a) Superblock
 - (b) Block
 - (c) Granule
 - (d) Group
- 9.18 Complete the following table by determining the actual bits to be transmitted to the decoder for the given sets of three normalized subband sample values.

Subband samples	<i>bspl</i>	Code word
−0.1106, 0.8436, −0.1886	5	
0.2309, 0.4764, 0.8709	7	
0.5839, −0.6475, 0.8338	9	
−0.1795, −0.2943, −0.7222	10	

- 9.19 Calculate the number of slots per frame for a Layer II coder with an input sampling rate of 48 kHz and output bitrate of 128 kbit/s.

- 9.20 With the aid of a diagram describe the format of an MPEG-1 Layer II audio frame.
- 9.21 Determine the actual bits required in the header of an MPEG-1 audio frame for the audio stream with the following configuration:

Property	Value
ID	ISO/IEC 11172-3
Layer	I
Error protection redundancy	None
Bitrate	128 kbit/s
Sampling frequency	44.1 kHz
Padding slot required	No
Mode	Single channel
Copyright	Protected
Original	Yes
Emphasis	None

- 9.22 Determine the actual bits required in the header of an MPEG-1 audio frame for the audio stream with the following configuration:

Property	Value
ID	ISO/IEC 11172-3
Layer	II
Error protection redundancy	None
Bitrate	256 kbit/s
Sampling frequency	48 kHz
Padding slot required	No
Mode	Intensity_stereo
Subbands in intensity_stereo	4–31
Copyright	Protected
Original	Yes
Emphasis	None

- 9.23 Decode the following bit stream that represents a single audio frame from an MPEG-1 Layer II bit stream.
- 9.24 With the aid of diagrams, explain how a multichannel audio signal can be encoded with an MPEG-2 encoder to produce an MPEG-1 compatible audio frame.

MATLAB EXERCISE 9.1

The aim of this exercise is to implement the Psychoacoustic model used in the MPEG-1 Layer I/II encoder.

1. Write a MATLAB function to determine the scalefactor for each subband given a block of 12 sets of 32 subband samples and using the scalefactor values defined in Section 9.1.2.
2. Write a MATLAB function to calculate the FFT of a set of 512 input audio samples using the equation defined in Section 9.1.3.1.

3. Write a MATLAB function to determine the maximum sound pressure level in each subband given a set of 256 FFT coefficients and the scalefactors for each subband of the corresponding subband samples using the equation defined in Section 9.1.3.2.
4. Write a MATLAB function to determine the threshold in quiet at the sub-sampled frequency values for each input sampling frequency.
5. Write a MATLAB function to locate the tonal and nontonal masking components for a set of 256 FFT coefficients using the code given in Section 9.1.3.4.
6. Write a MATLAB function to decimate the tonal and nontonal masking components for a set of 256 FFT coefficients using the code given in Section 9.1.3.5.
7. Write a MATLAB function to calculate the individual masking threshold for a set of tonal and nontonal masking components using the code given in Section 9.1.3.6.
8. Write a MATLAB function to calculate the global masking threshold for a set of individual masking thresholds using the equation given in Section 9.1.3.7.
9. Write a MATLAB function to calculate the minimum masking threshold in each subband for a given global masking thresholds using the equation given in Section 9.1.3.8.
10. Write a MATLAB function to calculate the signal-to-mask ratio for a set of minimum masking threshold and sound pressure level values using the pseudocode given in Section 9.1.3.9.
11. Combine these functions to produce a MATLAB program that calculates the signal-to-mask-ratio for each subband given a block of 12 sets of 32 subband samples.
12. Write a MATLAB function to calculate the number of bits allocated to each subband given a set of signal-to-mask ratio values for each subband.
13. Combine the functions from 11 and 12 and determine the number of bits allocated to each subband for a block of 12 sets of 32 subband samples.

MATLAB EXERCISE 9.2

The aim of this exercise is to implement the quantization and inverse quantization algorithms used in the MPEG Layer I encoder.

1. Write a MATLAB function to quantize a block of subband sample values with the number of bits specified by a set of bit allocation values for each subband using the quantization algorithm given in Section 9.1.6.
2. Write a MATLAB function to produce a block of subband samples values from a bit stream containing the code words corresponding to a set of quantized subband sample values.

MATLAB EXERCISE 9.3

The aim of this exercise is to implement the scalefactor coding and decoding algorithms used in the MPEG-1 Layer II encoder and decoder.

1. Write a MATLAB function to determine the two scalefactor difference classes for a set of three scalefactor indices using the algorithm described in Section 9.2.2.
2. Write a MATLAB function to determine the scalefactor to be used in the encoder and the scalefactor selection information given a set of three scalefactor indices and their scalefactor difference classes.
3. Write a MATLAB function to produce a set of three scalefactor indices given a bit stream containing the scalefactors and scalefactor selection information for an MPEG-1 Layer II audio frame.

REFERENCES

1. ISO/IEC 11172-3: 1993, Information technology—Coding of moving pictures and associated audio for digital storage media at up to about 1.5 Mbit/s, Part 3: Audio.
2. ITU-R Recommendation BS-775-1, Multichannel stereophonic sound system with and without accompanying picture.
3. ISO/IEC 13818-3: 1995/Amd. 1: 1996, Information technology—Generic coding of moving pictures and associated audio information, Part 3: Audio.
4. ISO/IEC 13818-7: 1997/Cor. 1: 1998, Information technology—Generic coding of moving pictures and associated audio information, Part 7: Advanced audio coding.